

Desenvolvimento do Painel de Administração da Plataforma de Crowdfunding JustCauses

OnlyHighIQ

2025/2026

1230900 Tiago Pedroso Silva

Desenvolvimento do Painel de Administração da Plataforma de Crowdfunding JustCauses

OnlyHighIQ

2025/2026

1230900 Tiago Pedroso Silva



Licenciatura em Engenharia Informática

Junho de 2026

Orientador ISEP: **Paulo Maio**

Supervisor Externo: **Carlos Lima**

Declaração de Integridade

Declaro ter conduzido este trabalho académico com integridade.

Não plagiei ou apliquei qualquer forma de uso indevido de informações ou falsificação de resultados ao longo do processo que levou à sua elaboração.

Portanto, o trabalho apresentado neste documento é original e de minha autoria, não tendo sido utilizado anteriormente para nenhum outro fim.

Declaro ainda que tenho pleno conhecimento do Código de Conduta Ética do P.PORTO.

ISEP, 11 de maio de 2026

Resumo

Este relatório descreve o trabalho realizado no âmbito da unidade curricular de Projeto/Estágio da Licenciatura em Engenharia Informática do ISEP, durante o estágio na OnlyHighIQ, centrado no desenvolvimento do painel de administração da plataforma de *crowdfunding* JustCauses. O objetivo principal consistiu em conceber e implementar uma solução interna que permitisse às equipas de operação gerir utilizadores, campanhas, categorias, processos de verificação de identidade, levantamentos, transações e denúncias, com autenticação centralizada, controlo de acesso por perfis e registo de auditoria.

O trabalho iniciou-se com a análise do domínio e com o desenho da arquitetura da solução, suportada por um *front-end* em React e um *back-end* em Node.js/Express com TypeScript, assentes em PostgreSQL, DynamoDB e vários serviços da AWS, nomeadamente Cognito, S3, SES e CloudWatch. A implementação foi organizada por *sprints* e incidiu tanto sobre módulos transversais do ecossistema, como autenticação, gestão de roles, *page gates* e auditoria, como sobre módulos específicos da JustCauses, incluindo revisão de campanhas, gestão de categorias, *analytics*, moderação de conteúdo reportado, fila de KYC, levantamentos, transações e administração de contas internas.

No final do período coberto por este relatório, a solução apresenta uma base funcional sólida e cobre a maioria dos requisitos nucleares definidos para o painel, embora alguns módulos permaneçam parciais ou por concluir, sobretudo nas áreas de suporte, detalhe de utilizadores e decisão de pedidos KYC. O estágio permitiu consolidar competências de desenvolvimento *full-stack*, desenho arquitetural, integração com serviços *cloud* e trabalho iterativo em contexto real de produto.

Palavras-chave: Backoffice, Administração de Plataforma, Crowdfunding, JustCauses, RBAC, AWS

Abstract

This report describes the work carried out as part of the Project/Internship course of the BSc in Informatics Engineering at ISEP, during an internship at OnlyHighIQ, focused on the development of the administration panel for the JustCauses crowdfunding platform. The main goal was to design and implement an internal solution that would allow operations teams to manage users, campaigns, categories, identity verification processes, withdrawals, transactions, and reported content, with centralized authentication, role based access control, and audit logging.

The work began with domain analysis and architecture design. The solution is based on a React front end and a Node.js/Express back end written in TypeScript, supported by PostgreSQL, DynamoDB, and several AWS services, namely Cognito, S3, SES, and CloudWatch. Implementation was organized in sprints and covered both cross platform modules, such as authentication, role management, page gates, and auditing, and JustCauses specific modules, including campaign review, category management, analytics, reported content moderation, the KYC queue, withdrawals, transactions, and internal admin account management.

By the end of the period covered by this report, the solution provides a solid functional foundation and covers most of the core requirements defined for the panel, although some modules remain partial or unfinished, especially in the areas of support, user detail views, and KYC decision workflows. The internship also strengthened practical skills in full stack development, software architecture, cloud service integration, and iterative delivery in a real product environment.

Agradecimentos

A realização deste estágio e deste relatório beneficiou do acompanhamento e da disponibilidade de várias pessoas, a quem deixo um agradecimento sincero. Ao Professor Paulo Maio, pelo acompanhamento académico e pelas observações feitas ao longo do trabalho. Ao Carlos Lima e à equipa da OnlyHighIQ, pela integração num contexto real de desenvolvimento, pela partilha de conhecimento e pela confiança dada durante a implementação do backoffice da JustCauses. Agradeço também aos colegas com quem trabalhei durante o estágio, pelo apoio diário, pela entreaajuda nas fases de análise, implementação e validação, e pelo contexto técnico e funcional que tornou possível avançar com o projeto de forma consistente.

Índice

1	Introdução	1
1.1	Enquadramento/Contexto	1
1.2	Descrição do Problema	2
1.2.1	Objetivos	2
1.2.2	Abordagem	3
1.2.3	Contributos	3
1.2.4	Planeamento do trabalho	4
1.3	Estrutura do relatório	5
2	Estado da Arte	7
2.1	Trabalhos relacionados	7
2.1.1	Shopify Admin	7
2.1.2	Stripe Dashboard	9
2.1.3	Directus	10
2.1.4	Appsmith	11
2.1.5	Análise comparativa	13
2.2	Tecnologias existentes	13
2.2.1	Frameworks de <i>UI</i> para administração	13
2.2.2	Frameworks de <i>front-end</i>	14
2.2.3	Frameworks de <i>back-end</i>	14
2.2.4	Bases de dados relacionais	15
2.2.5	Bases de dados NoSQL e sistemas de auditoria	15
2.2.6	Provedores de serviços <i>cloud</i>	16
2.2.7	Autenticação e gestão de identidade	16
2.2.8	Pagamentos e transações <i>on-chain</i>	17
2.3	Análise e seleção das tecnologias	17
2.4	Conclusão	18
3	Análise e Desenho da Solução	21
3.1	Domínio do problema	21
3.2	Requisitos funcionais e não funcionais	22
3.2.1	Requisitos funcionais	22
	Autenticação, autorização e administração	22
	Operação da plataforma	23
	Rastreabilidade	23
3.2.2	Requisitos não funcionais	23
3.3	Casos de utilização	24
3.4	Arquitetura da solução	25
3.4.1	Nível 1 — Contexto do sistema	25
3.4.2	Nível 2 — Contentores	25
3.4.3	Nível 3 — Componentes	26

3.4.4	Vista física	28
3.5	Modelo de permissões	29
3.6	Modelo de dados	30
3.7	Diagramas de sequência do sistema	30
3.7.1	Resolução de permissões e acesso a páginas	30
3.7.2	Autenticação (BOSH-26)	32
3.7.3	Revisão de campanhas (BOSH-1)	33
3.7.4	Gestão de categorias (BOSH-7)	34
3.7.5	Aprovação de levantamentos (BOSH-5)	35
3.8	Alternativas ao design	36
3.9	Conclusão	37
4	Implementação da Solução	39
4.1	Ambiente e estrutura do projeto	39
4.2	Autenticação e controlo de acesso	39
4.2.1	Passo 1 — autenticar e resolver o contexto do operador	40
4.2.2	Passo 2 — decidir o acesso à aplicação e à página	40
4.2.3	Passo 3 — limitar ações dentro da interface	40
4.2.4	Passo 4 — revalidar cada pedido no servidor	41
4.3	Módulos transversais ao ecossistema	42
4.3.1	Seletor de aplicações	42
4.3.2	Configuração administrativa do acesso	42
4.3.3	Sistema de auditoria	42
4.4	Módulos operacionais da JustCauses	43
4.4.1	Dashboard e métricas	43
4.4.2	Gestão de campanhas	43
4.4.3	Gestão de categorias	44
4.4.4	<i>Analytics</i>	44
4.4.5	Aprovação de levantamentos	44
4.4.6	Transações	44
4.5	Testes	45
4.5.1	Validação manual por módulo	45
4.5.2	Testes de integração da API da JustCauses	45
4.5.3	Testes unitários ao módulo de eventos de domínio	46
4.5.4	Limitações e trabalho futuro em testes	46
4.6	Avaliação da solução	46
4.7	Conclusão	48
5	Conclusões	49
5.1	Objetivos concretizados	49
5.2	Limitações e trabalho futuro	50
5.2.1	Limitações identificadas	50
5.2.2	Trabalho futuro	50
5.3	Apreciação final	51
	Bibliografia	53
A	Diagramas de sequência do sistema — Versão detalhada	55
A.1	Autenticação (BOSH-26)	56
A.2	Revisão de campanhas (BOSH-1)	57

A.3	Gestão de categorias (BOSH-7)	58
A.4	Aprovação de levantamentos (BOSH-5)	59

Lista de Figuras

2.1	Interface de gestão de encomendas do Shopify Admin [4]	8
2.2	Interface de gestão de transações do Stripe Dashboard [6]	9
2.3	Interface de gestão de dados do Directus [7]	11
2.4	Interface de construção de ferramentas internas no Appsmith [8]	12
3.1	Diagrama de contentores — C4 Nível 2	26
3.2	Diagrama de componentes do <i>back-end</i> — C4 Nível 3	27
3.3	Diagrama de componentes do <i>front-end</i> — C4 Nível 3	28
3.4	SSD — Autenticação (BOSH-26)	32
3.5	SSD — Revisão de campanhas (BOSH-1)	33
3.6	SSD — Gestão de categorias (BOSH-7)	34
3.7	SSD — Aprovação de levantamentos (BOSH-5)	35
A.1	SSD detalhado — Autenticação (BOSH-26)	56
A.2	SSD detalhado — Revisão de campanhas (BOSH-1)	57
A.3	SSD detalhado — Gestão de categorias (BOSH-7)	58
A.4	SSD detalhado — Aprovação de levantamentos (BOSH-5)	59

Lista de Tabelas

1.1	Missão, Visão e Valores da OnlyHighIQ	1
1.2	Planeamento do trabalho	5
2.1	Comparação dos sistemas de administração analisados	13
2.2	Comparação de <i>frameworks</i> de <i>UI</i> para painéis de administração	14
2.3	Comparação de <i>frameworks</i> de <i>front-end</i> para painéis de administração	14
2.4	Comparação de <i>frameworks</i> de <i>back-end</i> para Application Programming Interfaces (APIs) REST	15
2.5	Comparação de sistemas de gestão de bases de dados relacionais	15
2.6	Comparação de soluções para auditoria e dados de alta escrita	16
2.7	Comparação de provedores de serviços <i>cloud</i> por área de serviço	16
2.8	Comparação de soluções de autenticação e gestão de identidade	17
2.9	Comparação de soluções de pagamento para plataformas globais de <i>crowdfunding</i>	17
3.1	Principais entidades do domínio do painel de administração	22
3.2	Requisitos não funcionais do painel de administração	24
3.3	Serviços AWS e respetivo papel na solução	28
3.4	Camadas de aplicação do modelo de autorização	29
4.1	Módulos cobertos pela suite de testes de integração da JustCauses API	45
4.2	Estado de implementação por eixo funcional	47
5.1	Grau de concretização dos objetivos	49

Lista de Acrónimos

API	Application Programming Interface.
AWS	Amazon Web Services.
CI/CD	Continuous Integration / Continuous Deployment.
CRUD	Create, Read, Update, Delete.
CSV	Comma-Separated Values.
E2E	End to End.
HTTP	Hypertext Transfer Protocol.
ISEP	Instituto Superior de Engenharia do Porto.
KPI	Key Performance Indicator.
KYC	Know Your Customer.
LEI-PROJ	Projeto em Licenciatura em Engenharia Informática.
MFA	Multi-Factor Authentication.
MVP	Minimum Viable Product.
OJC	Only Just Causes.
PDF	Portable Document Format.
PostgreSQL	PostgreSQL Relational Database.
RBAC	Role-Based Access Control.
REST	Representational State Transfer.
SES	Simple Email Service.
SSD	System Sequence Diagram.
SSE	Server-Sent Events.
TTL	Time To Live.
UC	Unidade Curricular.
UI	User Interface.

Capítulo 1

Introdução

Neste capítulo inicial é apresentado o enquadramento do projeto, o contexto em que surgiu e o problema que procura resolver. São ainda descritos os objetivos definidos, a abordagem seguida para os atingir, os contributos esperados e o planeamento estabelecido para a execução do trabalho. O capítulo termina com uma apresentação sucinta da estrutura do restante relatório.

1.1 Enquadramento/Contexto

O presente projeto desenvolve-se no âmbito da Unidade Curricular (UC) Projeto em Licenciatura em Engenharia Informática (LEI-PROJ) da Licenciatura em Engenharia Informática do Instituto Superior de Engenharia do Porto (ISEP) e decorre nas instalações da empresa *OnlyHighIQ*, sediada no Porto, com início a 23 de fevereiro de 2026.

A *OnlyHighIQ* é uma empresa de inovação tecnológica recentemente fundada, instalada no centro do Porto, numa zona de fácil acesso a parceiros e clientes. A empresa tem como objetivo principal construir um ecossistema de plataformas digitais que, através da tecnologia, permitam a pessoas de todo o mundo aceder a serviços e indústrias a que de outra forma não teriam acesso, suportado por um ecossistema próprio de transações financeiras. A missão, a visão e os valores que orientam esta ambição estão sintetizados na Tabela 1.1. [1]

Tabela 1.1: Missão, Visão e Valores da *OnlyHighIQ*

Missão	<i>"To create a dynamic community where brilliant minds collaborate to develop cutting-edge solutions and innovative platforms that transform visionary ideas into reality."</i> [1]
Visão	<i>"To create a limitless innovation landscape where the most brilliant minds, have the tools to build the future—ensuring that progress and opportunity are distributed to everyone, everywhere."</i> [1]
Valores	<i>"Innovation Without Borders, Collaborative Excellence, Impact with Purpose, Courage to Disrupt, Principle-Led Integrity"</i> [1]

O ecossistema da *OnlyHighIQ* é composto por um conjunto de aplicações independentes que partilham uma camada de identidade global e um módulo de pagamentos comum. A primeira aplicação em desenvolvimento ativo é a **JustCauses** — uma plataforma de *crowdfunding* orientada ao apoio de causas sociais a nível global, já com uma base de código existente. Estão ainda planeadas futuras aplicações para o ecossistema, nomeadamente a *Only High Value* (investimento fracionado em ativos de valor elevado) e a *Only Second Hand* (moda em segunda mão). [2]

O trabalho realizado neste estágio centra-se no desenvolvimento do **painel de administração** da plataforma JustCauses — um sistema interno pensado para operar a plataforma de forma controlada, acompanhar campanhas e movimentos financeiros, configurar o acesso dos operadores e manter rastreabilidade sobre as ações mais sensíveis. Algumas áreas complementares do produto, como verificação de identidade, denúncias de conteúdo ou gestão detalhada de utilizadores, existem no ecossistema do painel, mas surgem neste relatório apenas como enquadramento funcional, sem aprofundamento próprio. A arquitetura do painel foi concebida de forma compatível com a especificação de *backoffice* do ecossistema da *OnlyHighIQ*, o que possibilitará a sua integração futura num sistema de administração centralizado sem necessidade de refatoração estrutural. A solução está alojada na Amazon Web Services (AWS) e é desenvolvida em React com TypeScript e Vite no *front-end* e em Node.js com Express e TypeScript no *back-end*, com persistência em PostgreSQL Relational Database (PostgreSQL) e integração de múltiplos serviços AWS.

1.2 Descrição do Problema

Uma plataforma de *crowdfunding* que opere a nível global enfrenta desafios operacionais que vão muito além da experiência do utilizador final. Não basta publicar campanhas e receber doações: é preciso rever submissões, acompanhar pedidos de levantamento, perceber o estado da operação e garantir que cada membro da equipa interna acede apenas ao que lhe compete.

Na ausência de um painel de administração dedicado, estas operações acabam por depender de processos manuais pouco rastreáveis: a aprovação de campanhas não segue um fluxo claro, os pedidos de levantamento não têm acompanhamento consistente, a informação necessária para decidir fica dispersa e as ações dos operadores internos não são registadas de forma uniforme. O resultado é simples: mais atraso, mais margem para erro e menos capacidade para perceber o que aconteceu quando surge um problema. Alguns fluxos adjacentes do produto, como a verificação de identidade ou a análise de conteúdo reportado, sofrem das mesmas limitações, embora não sejam o foco central deste documento.

A **JustCauses**, estando ainda em fase de Minimum Viable Product (MVP), necessita por isso de um painel de administração que centralize a operação da plataforma e dê às equipas internas um modo de trabalho mais controlado e auditável. Neste contexto, o controlo de acesso deixa de ser um detalhe técnico e passa a ser parte da própria solução: o painel tem de distinguir aplicações, páginas, ações e perfis de operador sem perder coerência entre *front-end* e *back-end*. Ao mesmo tempo, a arquitetura deve manter a separação entre capacidades transversais ao ecossistema e módulos específicos da JustCauses, para que a base criada agora possa ser reutilizada por futuras aplicações da *OnlyHighIQ*.

1.2.1 Objetivos

Para dar resposta ao problema identificado, o trabalho desenvolvido durante o estágio foi orientado por três linhas principais:

Operação da plataforma:

- **Campanhas e categorias:** suportar a revisão de campanhas submetidas e a gestão das categorias que organizam o catálogo da plataforma.

- **Fluxos financeiros:** disponibilizar informação operacional sobre transações e apoiar a análise e processamento de pedidos de levantamento.
- **Visão global da operação:** fornecer um *dashboard* com indicadores úteis para leitura rápida do estado da plataforma.

Controlo de acesso e auditoria:

- **Autenticação e contexto do operador:** integrar a autenticação com Cognito e resolver, após *login*, o perfil efetivo de cada operador no painel.
- **Autorização transversal:** aplicar permissões ao nível da aplicação, da página, das ações visíveis na interface e dos endpoints expostos pela Application Programming Interface (API).
- **Rastreabilidade:** registar ações relevantes e tentativas de acesso falhadas para reforçar a auditabilidade do sistema.

Arquitetura, qualidade e evolução:

- **Arquitetura extensível:** garantir uma base compatível com a evolução para um *backoffice* multi-aplicação no ecossistema da *OnlyHighIQ*.
- **Garantia de qualidade:** reforçar a fiabilidade da solução através de testes manuais, testes automatizados e uma cadeia de entrega contínua.
- **Interfaces de trabalho:** desenvolver páginas *web* pensadas para utilização interna, com navegação clara e foco na eficiência operacional.

Outros módulos presentes no produto mantêm-se neste relatório como enquadramento funcional do painel, mas não constituem o eixo principal da análise apresentada.

1.2.2 Abordagem

O desenvolvimento do painel segue uma metodologia ágil assente no **SCRUM**, adotada pela *OnlyHighIQ* como forma de estruturar o trabalho em equipa e garantir entregas incrementais de valor. O SCRUM organiza o trabalho em *sprints* de duração fixa e assenta em três pilares fundamentais: **Transparência, Inspeção e Adaptação**, promovendo uma comunicação constante entre os membros da equipa. [3] As tarefas são geridas no **Jira**, onde os *sprints* são definidos pelo supervisor, com distribuição de trabalho e acompanhamento do progresso de cada membro.

Do ponto de vista técnico, o painel de administração é uma aplicação exclusivamente *web* — não existe componente móvel neste sistema, cujo acesso é restrito a operadores internos. A solução assenta numa separação clara entre *front-end* e *back-end*. O *front-end* é desenvolvido em React com TypeScript e Vite, com recurso ao Bootstrap para estilos. O *back-end* utiliza Node.js com Express e TypeScript, com persistência em PostgreSQL e integração de múltiplos serviços AWS — nomeadamente Cognito para autenticação, S3 para armazenamento de ficheiros, DynamoDB para dados de auditoria e CloudWatch para monitorização. A adoção de práticas de Continuous Integration / Continuous Deployment (CI/CD) e de controlo de versões com *branching* faz parte do fluxo de trabalho estabelecido pela empresa.

1.2.3 Contributos

Os contributos esperados deste estágio organizam-se em três dimensões:

Tecnológicos: implementação do painel de administração da **JustCauses**, com foco nos módulos operacionais mais maduros, no mecanismo transversal de autorização, na auditoria de ações internas e na consulta de informação financeira e analítica; arquitetura modular compatível com a especificação de *backoffice* do ecossistema; e adoção de práticas de CI/CD e testes automatizados.

Organizacionais: a *OnlyHighIQ* passa a dispor das ferramentas operacionais necessárias para gerir a plataforma **JustCauses** de forma controlada e auditável, o que é condição indispensável para a abertura ao público. A arquitetura modular adotada facilita a incorporação futura de outras aplicações do ecossistema no mesmo sistema de administração.

Pedagógicos: aplicação prática de conhecimentos em desenvolvimento *full-stack*, sistemas de autorização e controlo de acesso, arquitetura *cloud* e metodologias ágeis, numa equipa profissional e num contexto real de produto.

Síntese dos contributos previstos:

- Consolidação de um painel interno para a operação diária da JustCauses.
- Implementação de um modelo de acesso coerente entre *login*, navegação, interface e API.
- Suporte à revisão de campanhas, gestão de categorias e consulta de informação financeira.
- Sistema de auditoria para ações relevantes e tentativas de acesso recusadas.
- *Dashboard* com métricas operacionais e Key Performance Indicators (KPIs) da plataforma.
- Arquitetura extensível e compatível com o crescimento do ecossistema da *OnlyHighIQ*.
- Aplicação de boas práticas de engenharia ao longo de todo o ciclo de desenvolvimento.

1.2.4 Planeamento do trabalho

O estágio teve início a 23 de fevereiro de 2026. Numa fase inicial, o foco incide na integração na equipa, na compreensão do código existente e no planeamento das funcionalidades a desenvolver. O trabalho subsequente é organizado em *sprints* pelo supervisor externo, Carlos Lima, com recurso ao Jira para distribuição e acompanhamento de tarefas.

Tabela 1.2: Planeamento do trabalho

Período	Tarefa
23/02/2026 – 03/03/2026	Fase de integração, análise do código existente e planeamento
04/03/2026 – 18/03/2026	Sprint 1 — Infraestrutura e DevOps; autenticação; base do modelo de autorização
18/03/2026 – 01/04/2026	Sprint 2 — Seletor de aplicações; aplicação de permissões por aplicação e por página; auditoria de acessos
01/04/2026 – 15/04/2026	Sprint 3 — Configuração administrativa do acesso; consolidação de operadores internos; revisão de campanhas
15/04/2026 – 29/04/2026	Sprint 4 — Gestão de categorias; validação de conteúdos submetidos; estabilização de fluxos operacionais
29/04/2026 – 13/05/2026	Sprint 5 — Processamento de levantamentos; salvaguardas operacionais e tratamento de exceções
13/05/2026 – 27/05/2026	Sprint 6 — Dashboard de analytics; observabilidade; consolidação de logs
27/05/2026 – 10/06/2026	Sprint 7 — Ajustes finais, testes e estabilização de módulos complementares

1.3 Estrutura do relatório

Para além da Introdução, este documento contém mais 4 capítulos, que acompanham as várias etapas do trabalho desenvolvido.

No Capítulo 2 é apresentado o estado da arte, com análise de sistemas de administração representativos do estado atual da indústria e comparação das tecnologias relevantes para o desenvolvimento de painéis de administração de plataformas digitais.

No Capítulo 3 descreve-se a análise e o desenho da solução: arquitetura do painel definida, requisitos funcionais e não funcionais, modelo de permissões, modelo de dados e as principais decisões de desenho tomadas.

No Capítulo 4 detalham-se os processos de implementação, com exemplos dos módulos desenvolvidos, testes realizados para validação da solução e imagens da interface do painel de administração.

No Capítulo 5 são apresentadas as conclusões, com uma reflexão sobre os resultados obtidos, as limitações encontradas e as perspetivas de trabalho futuro.

Capítulo 2

Estado da Arte

O desenvolvimento de um painel de administração, também designado por *backoffice*, exige uma análise diferente da que seria feita para uma aplicação pública. Estes sistemas são utilizados por operadores internos, têm acesso restrito e, por isso, raramente disponibilizam publicamente todos os seus fluxos, ecrãs e regras de funcionamento. Esta limitação condiciona a análise de trabalhos relacionados: em vez de procurar apenas produtos equivalentes com demonstrações completas, torna-se mais adequado estudar padrões observáveis em painéis de administração, ferramentas de construção de aplicações internas e recomendações técnicas para autorização e auditoria.

Neste capítulo analisam-se, primeiro, as capacidades mais relevantes para um painel de administração seguro e auditável. Em seguida, apresentam-se sistemas e ferramentas relacionados, distinguindo painéis comerciais, plataformas configuráveis e *frameworks* de administração. Por fim, enquadram-se as tecnologias usadas no projeto, separando as escolhas já existentes na empresa das decisões relevantes para o trabalho desenvolvido durante o estágio.

2.1 Critérios de análise

O painel desenvolvido neste estágio não corresponde a uma aplicação de consumidor final. O seu objetivo é apoiar operações internas da plataforma JustCauses, com controlo de acesso, rastreabilidade e integração com dados sensíveis. Assim, a análise do estado da arte deve privilegiar critérios ligados à operação e à segurança, e não apenas aspetos visuais ou escolhas genéricas de *front-end*.

A Tabela ?? apresenta os critérios usados para comparar os sistemas e ferramentas analisados neste capítulo.

Tabela 2.1: Critérios de análise para painéis de administração

Critério	Relevância para o projeto
Controlo de acesso	O painel deve distinguir operadores, perfis, aplicações, páginas e ações, aplicando permissões no <i>front-end</i> e no <i>back-end</i> .
Auditoria	As operações sensíveis devem deixar registos consultáveis, com identificação do operador, ação executada, entidade afetada e data.
Fluxos operacionais	O sistema deve suportar processos internos, como revisão de campanhas, gestão de categorias, consulta financeira e administração de acessos.
Adaptação ao domínio	A solução deve acomodar regras específicas da JustCauses e manter compatibilidade com a evolução do ecossistema da OnlyHighIQ.
Manutenibilidade	A arquitetura deve permitir evolução incremental, testes e integração com a base de código existente.

Estes critérios são também coerentes com recomendações de segurança aplicacional. A OWASP recomenda que a autorização seja aplicada no servidor e que o sistema negue o acesso por omissão quando não exista uma regra explícita de permissão **OWASPAuthorizationCheatSheet**. Além disso, o modelo Role-Based Access Control (RBAC) usado no projeto aproxima-se de abordagens mais contextuais quando combina perfil, aplicação, página e ação. Esta ideia é compatível com a definição de controlo de acesso baseado em atributos descrita pelo NIST, onde a autorização resulta da avaliação de atributos do sujeito, do objeto, da operação e, quando aplicável, do contexto **NISTABAC**.

2.2 Trabalhos relacionados

A análise de trabalhos relacionados considera dois grupos de soluções. O primeiro inclui painéis comerciais de plataformas consolidadas, úteis para observar padrões de operação, permissões e rastreabilidade. O segundo inclui ferramentas de administração configuráveis, relevantes para avaliar se faria sentido adotar uma solução pronta em vez de uma implementação customizada.

2.2.1 Painéis comerciais de referência

O Shopify Admin é um exemplo de painel comercial orientado a equipas operacionais sem perfil técnico. A documentação pública descreve um modelo de permissões associado a funções, com permissões ao nível da loja, organização e ponto de venda, dependendo do contexto e do plano contratado [5]. Este modelo é relevante porque demonstra a importância de separar áreas funcionais e tarefas sensíveis dentro do mesmo painel. No entanto, o Shopify Admin está associado ao domínio do comércio eletrónico, o que limita a sua utilidade como referência direta para fluxos de campanhas, moderação e gestão financeira próprios de uma plataforma de *crowdfunding*.

O Stripe Dashboard é uma referência particularmente útil para operações financeiras e gestão de equipa. A documentação da Stripe descreve a atribuição de funções a membros da equipa, a possibilidade de um utilizador acumular vários papéis e a existência de histórico de segurança com atividade dos membros da conta **StripeTeams**. As funções específicas do Stripe Identity, como *Identity Analyst* e *Identity View Only*, mostram uma separação operacional entre consulta e revisão de verificações de identidade **StripeUserRoles**. Esta abordagem

é relevante para o projeto porque evidencia a necessidade de controlar ações sensíveis por tipo de operação, e não apenas por acesso geral ao painel.

Apesar da sua relevância, estes painéis comerciais têm uma limitação comum: são produtos fechados. A documentação pública permite observar conceitos e padrões, mas não permite reutilizar a sua arquitetura, adaptar os seus módulos internos ou validar todos os fluxos de operação. Por esse motivo, a sua utilidade neste relatório é sobretudo comparativa.

2.2.2 Ferramentas configuráveis e *frameworks* de administração

O Directus representa uma abordagem orientada a dados. A ferramenta disponibiliza uma interface de administração sobre bases de dados existentes e permite definir permissões por coleção, ação, campo e política **DirectusPermissions**. Esta granularidade é relevante para sistemas em que a principal necessidade é administrar entidades de dados. No contexto deste projeto, porém, a limitação está no facto de a interface ser construída a partir do modelo de dados, enquanto o painel da JustCauses precisa de fluxos de negócio com estados, validações e decisões operacionais próprias.

O Appsmith segue uma abordagem *low-code* para criação de ferramentas internas. A documentação descreve a ligação a bases de dados e APIs, a construção de interfaces com componentes visuais e a escrita de lógica através de consultas e JavaScript **AppsmithDocs**. Esta solução pode acelerar protótipos e painéis internos simples. No entanto, quando a aplicação exige regras de autorização consistentes, auditoria própria e fluxos sensíveis testáveis, a dispersão de lógica por componentes e scripts pode dificultar a manutenção.

As *frameworks* React Admin, AdminJS e Refine.dev ocupam uma posição intermédia. O React Admin disponibiliza componentes e abstrações para aplicações Create, Read, Update, Delete (CRUD), com integração por *data providers* e suporte a autenticação e permissões **ReactAdminDocs**. O AdminJS permite gerir recursos associados a modelos, ferramentas de mapeamento objeto-relacional ou APIs através de adaptadores **AdminJSDocs**. O Refine.dev adota uma arquitetura *headless*, separando lógica de dados e apresentação, com foco em aplicações administrativas e ferramentas internas **RefineDocs**. Estas opções são tecnicamente adequadas quando o projeto parte do zero ou quando os fluxos seguem padrões de administração comuns. No caso em análise, a empresa já possuía uma base React, rotas, componentes, serviços e uma especificação própria de permissões, pelo que a adoção de uma destas ferramentas implicaria substituir parte relevante da arquitetura existente.

2.2.3 Análise comparativa

A Tabela ?? sintetiza as abordagens analisadas e a sua relevância para o projeto.

Tabela 2.2: Comparação de abordagens para painéis de administração

Abordagem	Exemplos	Contributo observado	Limitação para o projeto
Painel comercial fechado	Shopify Admin, Stripe Dashboard	Bons padrões de permissões, equipas, operações sensíveis e histórico de atividade.	Baixa adaptabilidade e acesso limitado à implementação interna.
Administração orientada a dados	Directus	Permissões granulares por entidade, campo e ação.	Menor adequação a fluxos de negócio específicos e estados operacionais complexos.
Ferramentas <i>low-code</i>	Appsmith	Prototipagem rápida de ferramentas internas ligadas a dados e APIs.	Lógica de negócio distribuída por componentes e scripts, com impacto na testabilidade.
<i>Frameworks</i> de administração	React Admin, AdminJS, Refine.dev	Aceleram listagens, formulários, autenticação e operações CRUD.	Introduzem dependências estruturais e podem obrigar a adaptar a arquitetura existente.
Implementação customizada	Solução adotada	Controlo total sobre autorização, auditoria, fluxos e integração com a base existente.	Maior esforço inicial de desenvolvimento e manutenção.

Da comparação resulta que nenhuma abordagem pronta responde integralmente ao conjunto de requisitos do projeto. As soluções comerciais ajudam a identificar padrões, mas não são reutilizáveis. As ferramentas configuráveis aceleram aplicações internas simples, mas perdem adequação quando o domínio exige autorização transversal, auditoria própria e fluxos de decisão específicos. Assim, a implementação customizada surge como a opção mais coerente com o contexto da empresa.

2.3 Princípios técnicos relevantes

Mais do que comparar tecnologias de forma genérica, o estado da arte deve identificar os princípios que condicionam o desenho de um painel de administração. No projeto desenvolvido, destacam-se três: autorização aplicada de forma consistente, auditoria operacional e separação entre ferramentas genéricas e implementação à medida.

2.3.1 Autorização e separação de responsabilidades

Num painel de administração, a autenticação apenas confirma a identidade do operador. A autorização determina o que esse operador pode fazer. Esta distinção é importante porque a interface pode esconder botões ou páginas, mas a decisão final tem de ocorrer no servidor. Caso contrário, bastaria alterar um pedido Hypertext Transfer Protocol (HTTP) para tentar executar uma operação não autorizada. Por esse motivo, a OWASP recomenda que os controlos de acesso sejam aplicados no servidor, com uma postura de negação por omissão **OWASPAuthorizationCheatSheet**.

No projeto, esta recomendação traduz-se em duas camadas complementares. O *front-end* usa permissões para orientar a navegação e reduzir ações visíveis ao operador. O *back-end*, por sua vez, valida permissões em cada endpoint sensível. Esta separação evita que a interface seja tratada como fonte de verdade para autorização.

2.3.2 Auditoria operacional

A auditoria é uma capacidade central em sistemas internos. Não basta saber que uma entidade mudou de estado; é necessário saber quem executou a ação, quando, sobre que entidade e com que resultado. Este requisito ganha importância em operações financeiras, revisão de conteúdo, gestão de utilizadores e configuração de permissões.

O Stripe Dashboard é um exemplo público de painel que expõe histórico de segurança associado à atividade dos membros da conta **StripeTeams**. No projeto JustCauses, a auditoria foi tratada como uma responsabilidade própria do painel, com registos imutáveis em DynamoDB. Esta opção é compatível com a necessidade de escrita frequente e consulta por eventos. O suporte a *Time to Live* do DynamoDB permite ainda gerir a retenção de registos quando existirem políticas de expiração definidas **DynamoDBTTL**.

2.3.3 Ferramenta genérica ou implementação customizada

A adoção de uma ferramenta genérica de administração deve ser avaliada de acordo com o grau de especificidade do domínio. Quando o problema consiste sobretudo em listar, criar, editar e remover entidades, soluções como React Admin, AdminJS, Refine.dev, Directus ou Appsmith podem reduzir o esforço inicial. Quando o domínio exige fluxos de aprovação, permissões configuráveis por aplicação e página, auditoria interna e integração com serviços já existentes, a vantagem inicial dessas ferramentas pode ser anulada pelo custo de adaptação.

No caso deste estágio, a implementação customizada não resulta de rejeição das ferramentas existentes. Resulta da combinação de três fatores: a empresa já possuía uma base de código em React e Node.js; o modelo de permissões tinha requisitos próprios; e o painel precisava de evoluir como parte do ecossistema OnlyHighIQ, não como aplicação administrativa isolada.

2.4 Tecnologias enquadradoras do projeto

Algumas tecnologias usadas no projeto não foram escolhidas durante o estágio. React, TypeScript, Node.js, Express, PostgreSQL e AWS já faziam parte da base técnica da empresa ou da arquitetura prevista para o ecossistema. Por esse motivo, não faria sentido apresentar uma comparação extensa entre React, Vue e Angular, ou entre Express, FastAPI e Spring Boot, como se a seleção tivesse sido feita de raiz no âmbito deste trabalho.

A decisão relevante foi avaliar se a base existente era adequada para consolidar um painel de administração seguro e evolutivo. No *front-end*, React com TypeScript e Vite oferece uma base suficiente para construir interfaces internas com componentes reutilizáveis, rotas protegidas e integração com serviços de autenticação. A simplicidade relativa das páginas do painel torna menos relevante uma comparação profunda entre *frameworks* de interface. O desafio principal não esteve na tecnologia visual, mas na coerência entre navegação, permissões e chamadas à API.

No *back-end*, Node.js com Express e TypeScript permite manter a mesma linguagem entre cliente e servidor, simplificando contratos, tipos e integração com bibliotecas do ecossistema *npm*. A persistência relacional em PostgreSQL é adequada para entidades transacionais da plataforma, enquanto o DynamoDB foi usado para dados de configuração e auditoria. Esta separação evita misturar dados operacionais críticos com registos de eventos e configurações administrativas.

A autenticação assenta no AWS Cognito. A documentação do serviço descreve a utilização de grupos em *user pools*, com inclusão dos grupos nos tokens através da *claim* `cognito:groups` **AWSCognitoGroups**. No entanto, o projeto não depende apenas desses grupos para decidir o acesso. Após o *login*, o sistema obtém o contexto interno do operador, incluindo perfil, aplicação e permissões efetivas. Esta decisão torna o modelo mais flexível do que uma autorização baseada apenas no token de autenticação.

2.5 Análise e seleção da abordagem

A abordagem adotada no projeto foi manter a base técnica existente e desenvolver o painel de forma customizada. Esta decisão é justificada por quatro motivos principais.

Primeiro, o painel precisava de respeitar a arquitetura já existente da empresa. A introdução de uma *framework* de administração completa, como React Admin ou AdminJS, obrigaria a adaptar rotas, serviços, componentes e contratos de API a uma abstração externa. O benefício seria maior num projeto iniciado de raiz.

Segundo, o requisito de autorização era transversal. O sistema não precisava apenas de esconder opções na interface; precisava de validar aplicação, página e ação no servidor, mantendo consistência entre o *front-end*, o *back-end* e a configuração administrativa de permissões.

Terceiro, a auditoria era parte do desenho da solução. A rastreabilidade de ações internas e tentativas de acesso negado exigia um modelo próprio, integrado com os serviços e entidades do painel.

Por fim, a solução tinha de ser extensível ao ecossistema da OnlyHighIQ. Um painel customizado permite separar módulos transversais, como autenticação, operadores, permissões e auditoria, de módulos específicos da JustCauses, como campanhas, categorias, transações e levantamentos.

2.6 Conclusão

A análise realizada mostra que o estado da arte dos painéis de administração deve ser observado sobretudo através de capacidades: controlo de acesso, auditoria, fluxos operacionais, integração com dados e manutenibilidade. A escassez de exemplos públicos completos não invalida a análise; apenas obriga a combinar documentação oficial, padrões observáveis e ferramentas comparáveis.

Os painéis comerciais, como Shopify Admin e Stripe Dashboard, mostram padrões relevantes de gestão de operadores, permissões e histórico de atividade, mas não são adaptáveis ao domínio do projeto. Ferramentas como Directus, Appsmith, React Admin, AdminJS e Refine.dev podem acelerar a criação de ferramentas internas, mas tornam-se menos adequadas quando o domínio exige regras próprias de autorização, auditoria e evolução arquitetural. Neste contexto, a implementação customizada sobre a base React, Node.js, PostgreSQL e AWS já existente foi a opção mais coerente. Esta conclusão fundamenta as decisões de análise e desenho apresentadas no capítulo seguinte.

Capítulo 3

Análise e Desenho da Solução

Neste capítulo descreve-se o processo de análise e o desenho da solução desenvolvida. O ponto de partida é a caracterização do domínio do problema, identificando os conceitos centrais e as relações entre eles. A partir daí, especificam-se os requisitos funcionais e não funcionais do painel de administração. Segue-se a apresentação da arquitetura da solução com recurso ao modelo C4 [9], e descrevem-se as principais decisões de estrutura: o modelo de controlo de acesso baseado em funções (RBAC) e o modelo de dados. O capítulo encerra com um resumo das decisões de desenho mais relevantes.

3.1 Domínio do problema

O painel de administração da JustCauses opera sobre um domínio composto por um conjunto de entidades relacionadas, cada uma com o seu ciclo de vida e as suas regras de negócio. A compreensão dessas entidades e das relações entre elas é o ponto de partida para qualquer decisão de arquitetura ou de desenho.

A Tabela 3.1 apresenta as principais entidades do domínio e uma descrição sucinta do seu papel na plataforma.

Tabela 3.1: Principais entidades do domínio do painel de administração

Entidade	Descrição
Utilizador	Conta registada na plataforma JustCauses, com estado (ativo, suspenso) e tipo de perfil associado (doador ou criador).
Criador	Utilizador com perfil de criador de campanhas, sujeito a processo de verificação de identidade (Know Your Customer (KYC)).
Campanha	Projeto de angariação de fundos submetido por um criador, com estados de ciclo de vida geridos pelo painel: pendente (PENDING), ativa (ACTIVE), inativa (INACTIVE), concluída (FINISHED), rejeitada (REJECTED) e em revisão de alterações (REVIEWING).
Categoria	Etiqueta temática associada a campanhas (ex.: Saúde, Educação, Ambiente), gerida exclusivamente pelo painel de administração.
Doação	Contribuição financeira de um utilizador para uma campanha, que pode ser sujeita a pedido de reembolso.
Levantamento	Pedido de transferência dos fundos angariados por um criador, sujeito a validação e aprovação manual pelo operador.
Documento KYC	Ficheiro de identificação submetido por um criador no âmbito do processo de verificação de identidade, com resultado de revisão associado.
Denúncia	Reporte de conteúdo inadequado (campanha ou comentário) submetido por um utilizador para revisão pelo operador do painel.
Ticket de suporte	Pedido de ajuda submetido por um doador ou criador, com histórico de mensagens e estado de resolução.
Registo de auditoria	Entrada imutável que documenta uma ação de um operador sobre uma entidade crítica da plataforma, com marca temporal e identificação do responsável.
Perfil de acesso	Conjunto de permissões atribuído a um operador interno, determinando quais as operações que lhe são permitidas no painel.

3.2 Requisitos funcionais e não funcionais

3.2.1 Requisitos funcionais

Os requisitos funcionais do painel de administração foram levantados a partir da especificação interna da empresa [2] e das discussões realizadas durante a fase de integração com a equipa. Para efeitos deste relatório, optou-se por apresentá-los com foco nas capacidades transversais que mais condicionaram a arquitetura e a implementação. Em vez de detalhar cada fluxo operacional isoladamente, privilegia-se aquilo que o sistema precisa de garantir para ser seguro, utilizável e evolutivo no contexto do ecossistema OnlyHighIQ.

Autenticação, autorização e administração

- **RF01** — O painel deve autenticar operadores internos através do AWS Cognito, com gestão de sessão e suporte a Multi-Factor Authentication (MFA).
- **RF02** — Após a autenticação, o sistema deve resolver o contexto interno do operador, incluindo perfil atribuído, aplicação a que pertence e permissões efetivas.

- **RF03** — O acesso ao backoffice de gestão e ao painel operacional da JustCauses deve ser separado por aplicação, impedindo a entrada em áreas não autorizadas.
- **RF04** — O sistema deve controlar o acesso a páginas com base em guardas de rota e em regras configuráveis por página.
- **RF05** — As permissões exigidas por cada página devem poder ser alteradas sem nova publicação da aplicação.
- **RF06** — Todos os endpoints sensíveis da API devem validar permissões no servidor antes de executar qualquer operação.
- **RF07** — O painel deve disponibilizar mecanismos de gestão administrativa para contas internas, perfis de acesso e configuração do modelo de autorização.

Operação da plataforma

- **RF08** — O painel deve disponibilizar um *dashboard* com métricas operacionais relevantes e indicadores de atividade da plataforma.
- **RF09** — O painel deve suportar a revisão do ciclo de vida das campanhas, incluindo estados de aprovação, rejeição e pedido de alterações.
- **RF10** — O painel deve permitir criar, editar e desativar categorias de campanhas.
- **RF11** — O painel deve disponibilizar consulta operacional de transações e pedidos de levantamento.
- **RF12** — A arquitetura deve permitir incorporar outros fluxos operacionais do ecossistema sem alterar o núcleo do mecanismo de autorização.

Rastreabilidade

- **RF13** — O painel deve manter um registo imutável das ações relevantes e das tentativas de acesso negado, consultável por filtros de data, operador e tipo de evento.

3.2.2 Requisitos não funcionais

Os requisitos não funcionais definem as condições de qualidade que o sistema deve satisfazer independentemente das funcionalidades específicas que implementa. A Tabela 3.2 sintetiza os principais requisitos identificados.

Tabela 3.2: Requisitos não funcionais do painel de administração

ID	Categoria	Descrição
RNF01	Segurança	Toda a comunicação deve ser feita sobre HTTPS. A autenticação é obrigatória em todos os pontos de entrada do painel, gerida pelo AWS Cognito com suporte a MFA.
RNF02	Rastreabilidade	Todas as operações de escrita realizadas por operadores devem ser registadas no sistema de auditoria, com marca temporal, identificador do operador e entidade afetada.
RNF03	Desempenho	As respostas da API para operações de leitura devem ter uma latência inferior a 500 ms em condições normais. O carregamento inicial do painel deve ocorrer em menos de 3 segundos.
RNF04	Usabilidade	A interface deve ser utilizável por operadores sem formação técnica, com navegação clara entre módulos e mensagens de confirmação e de erro explícitas.
RNF05	Disponibilidade	O sistema deve beneficiar dos níveis de disponibilidade da infraestrutura AWS, com monitorização ativa via CloudWatch.
RNF06	Manutenibilidade	O código deve seguir a arquitetura definida, com separação clara entre camadas e integração contínua configurada via CI/CD.
RNF07	Escalabilidade	O módulo de auditoria deve absorver picos de escrita sem degradar o desempenho da base de dados relacional principal.

3.3 Casos de utilização

Esta secção não aprofunda, de forma individual, os casos de utilização dos módulos de gestão de utilizadores, fila de KYC, *reports* e gestão de funções. Esses fluxos existem no produto, mas não são o ponto em que a solução se torna mais interessante do ponto de vista técnico. O aspeto decisivo é a forma como o sistema determina, em cada momento, o que um operador pode ver e fazer.

Em termos práticos, há dois grandes contextos de utilização: o backoffice de gestão e o painel operacional da JustCauses. Ambos partilham a mesma autenticação Cognito, mas o acesso efetivo é refinado por três decisões sucessivas: acesso à aplicação, acesso à página e autorização da ação pedida à API. Esta decomposição revelou-se mais útil para o desenho do sistema do que uma enumeração extensa de casos de uso por página, porque permitiu alinhar a interface, o *router*, o *middleware* da API e o sistema de auditoria sob a mesma regra de negócio.

Assim, os casos de utilização mais relevantes para o desenho da solução podem ser resumidos em quatro perguntas:

- O operador autenticado pode entrar na aplicação pedida?
- O operador pode abrir a página para a qual foi encaminhado?
- O operador pode executar a ação disponível na interface?
- O servidor deve aceitar ou rejeitar o pedido HTTP correspondente?

As respostas a estas perguntas são determinadas pelo modelo de permissões descrito na Secção 3.5 e pela configuração dinâmica de *page gates*, que passou a ser um dos elementos centrais do desenho.

3.4 Arquitetura da solução

A arquitetura do painel de administração foi definida com base nos requisitos levantados e nas restrições de infraestrutura já existentes no ecossistema da OnlyHighIQ. Para a descrever de forma estruturada, adotou-se o modelo C4 [9], que organiza a descrição em níveis de abstração crescente: contexto do sistema, contentores, componentes e código. Os três primeiros níveis são apresentados nas subsecções seguintes.

3.4.1 Nível 1 — Contexto do sistema

O painel de administração é um sistema de uso exclusivamente interno, acedido por operadores autorizados da OnlyHighIQ através de um navegador *web*. No que diz respeito ao contexto externo, o sistema interage com dois grupos de dependências: os serviços da infraestrutura AWS (Cognito para autenticação, S3 para armazenamento de ficheiros, SES para e-mail transaccional, DynamoDB para auditoria, Rekognition para moderação de conteúdo visual) e o ecossistema Coinbase CDP para o rastreamento e a gestão de transações financeiras *on-chain* em USDC. A base de dados PostgreSQL e os restantes módulos da plataforma JustCauses são partilhados entre o painel de administração e a aplicação de utilizador final.

3.4.2 Nível 2 — Contentores

O sistema é composto por dois contentores principais: a aplicação *front-end*, desenvolvida em React 19 com TypeScript e Vite, e a API *back-end*, desenvolvida em Node.js com Express 5 e TypeScript. A comunicação entre os dois é feita exclusivamente via Representational State Transfer (REST) sobre HTTPS, com autenticação baseada em tokens JWT emitidos pelo AWS Cognito e validados pelo *back-end* em cada pedido.

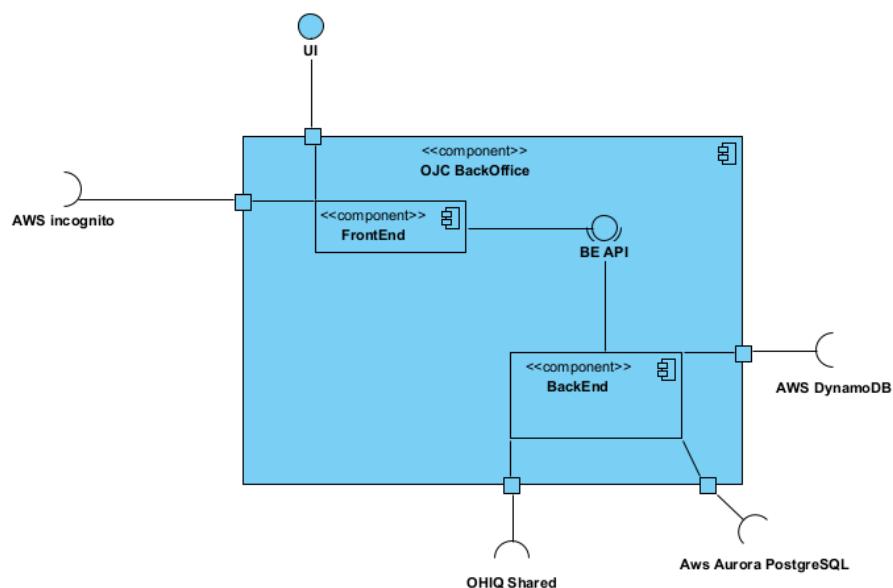


Figura 3.1: Diagrama de contêineres — C4 Nível 2

A camada de persistência é composta por duas bases de dados com responsabilidades distintas: o PostgreSQL para os dados transacionais da plataforma JustCauses (utilizadores, campanhas, doações, levantamentos e decisões de moderação) e o AWS DynamoDB para os dados de configuração e rastreabilidade do painel (roles, permissões, configuração de acesso por rota e registros de auditoria). O acesso a ficheiros (documentos de KYC, imagens de campanhas) é feito via AWS S3.

3.4.3 Nível 3 — Componentes

A API *back-end* é organizada em módulos funcionais correspondentes aos domínios operacionais do painel. Cada módulo segue uma estrutura de camadas inspirada em *Domain-Driven Design*: controladores HTTP que tratam o pedido e a resposta, serviços de domínio que contêm a lógica de negócio, repositórios que abstraem o acesso à base de dados, e objetos de transferência de dados (*DTOs*) que definem os contratos de comunicação com o exterior. Esta separação garante que a lógica de negócio não fica acoplada nem à camada de comunicação nem à de persistência, facilitando os testes e a evolução independente de cada camada.

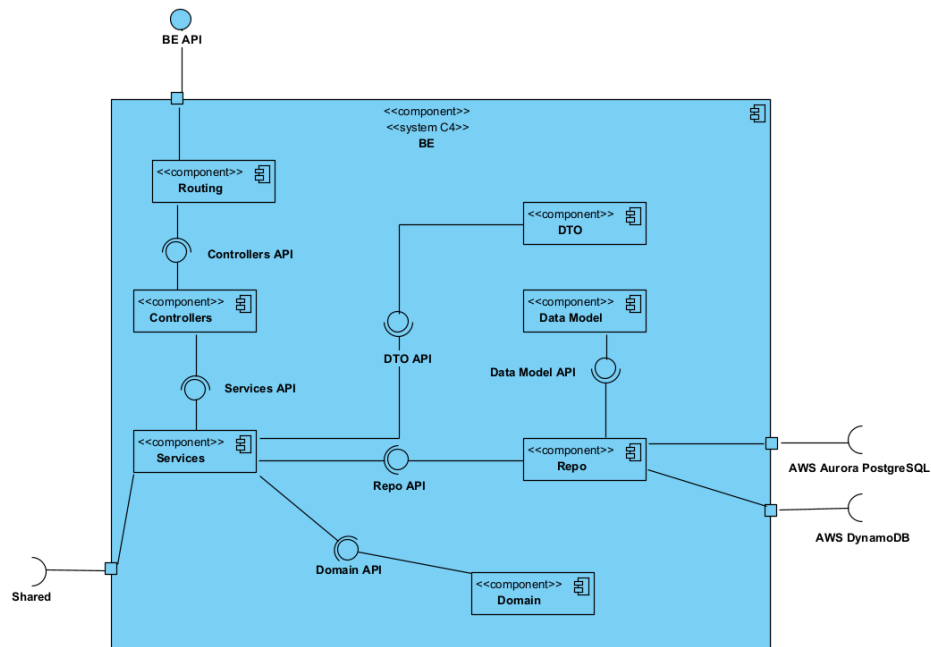
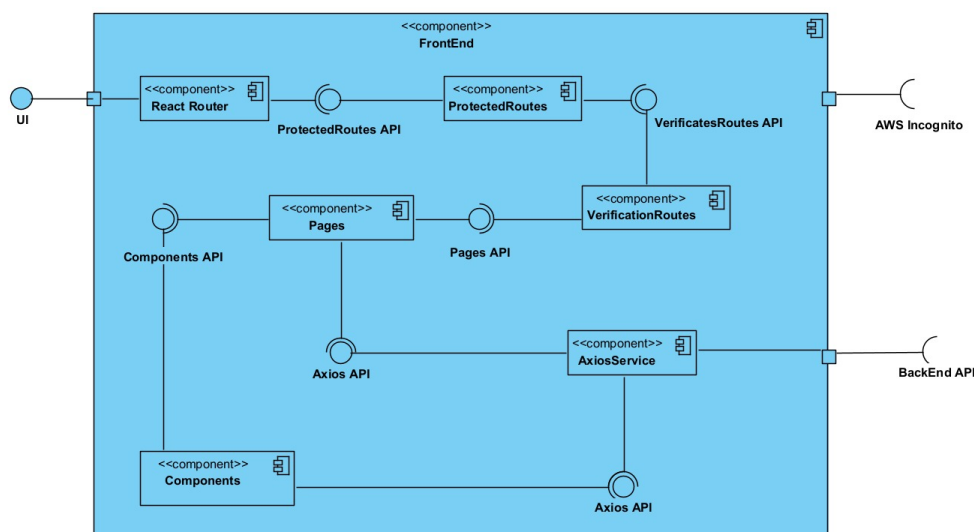


Figura 3.2: Diagrama de componentes do *back-end* — C4 Nível 3

O *front-end* é organizado em páginas e componentes React, com uma camada de serviços responsável pela comunicação com a API e uma camada de gestão de estado local. O roteamento é gerido pelo React Router, com proteção de rotas baseada no perfil de acesso do operador autenticado: uma rota protegida verifica o grupo do utilizador no token JWT antes de renderizar a página correspondente, redirecionando para o ecrã de acesso negado em caso de permissão insuficiente.

Figura 3.3: Diagrama de componentes do *front-end* — C4 Nível 3

3.4.4 Vista física

A infraestrutura do painel de administração reside integralmente na AWS, tirando partido do ecossistema de serviços geridos já adotado pelo ecossistema da *OnlyHighIQ*. A Tabela 3.3 identifica os serviços utilizados e o papel de cada um na solução.

Tabela 3.3: Serviços AWS e respetivo papel na solução

Serviço AWS	Papel na solução
Amazon Route 53	Resolução de nomes de domínio da aplicação.
Amazon CloudFront	Distribuição dos conteúdos estáticos do <i>front-end</i> com cache global e HTTPS.
Amazon S3 (UI)	Repositório dos <i>assets</i> estáticos compilados do <i>front-end</i> React.
Amazon S3 (Docs)	Armazenamento de documentos de KYC e ficheiros de campanhas submetidos pelos criadores.
Amazon Cognito	Gestão de identidade dos operadores internos do painel — autenticação com MFA, emissão de tokens JWT e grupos de utilizadores para controlo de acesso.
Application Load Balancer	Distribuição do tráfego HTTP para as instâncias do <i>back-end</i> Node.js.
Amazon EC2 / ECS	Alojamento do servidor <i>back-end</i> Node.js com Express, em contentor gerido ou em instância dedicada, dentro de uma <i>Virtual Private Cloud</i> (VPC).
Amazon RDS PostgreSQL	Base de dados relacional principal, partilhada com a JustCauses, que persiste utilizadores, campanhas, doações, levantamentos e verificações KYC.
Amazon DynamoDB	Tabela B0_Logs para o registo imutável de eventos de auditoria, com TTL configurável para gestão automática do ciclo de vida dos registos.
Amazon SES	Envio de notificações por e-mail a criadores e utilizadores em resposta a ações do painel (aprovação de campanha, rejeição de KYC, avisos de moderação).
Amazon CloudWatch	Monitorização de métricas de infraestrutura, alertas de erros e análise de logs do <i>back-end</i> .

O fluxo de uma operação típica percorre os seguintes serviços: o operador acede à interface *web* servida pelo **CloudFront** a partir dos *assets* em **S3**; o *front-end* autentica-se via **Cognito**

e obtém um token JWT; os pedidos API são encaminhados pelo **Application Load Balancer** para o servidor *back-end* em **EC2/ECS**; o servidor valida o token com o JWKS público do Cognito, verifica as permissões do operador e executa a operação sobre o **PostgreSQL**; o registo de auditoria é escrito de forma assíncrona no **DynamoDB**; e as notificações relevantes são enviadas via **SES**.

3.5 Modelo de permissões

O modelo de permissões não foi tratado apenas como uma lista de perfis. Na implementação real, a autorização resulta da combinação entre o contexto do operador, a aplicação em causa e a ação concreta pedida ao sistema. A autenticação é comum, mas a autorização é resolvida depois, com base nos dados internos do backoffice.

Quando o operador inicia sessão, o token do Cognito identifica a sessão, mas não chega para decidir o acesso às áreas internas. O *back-end* localiza o registo do utilizador interno, verifica se a conta está ativa, resolve o perfil atribuído e extrai desse perfil dois elementos centrais: `roleApplication`, que indica se o acesso pertence ao universo backoffice ou `just_causes`, e a lista de permissões efetivas. Esta decisão é importante porque o projeto não depende apenas do nome do perfil; depende do âmbito em que esse perfil pode operar.

A Tabela 3.4 resume as camadas em que a autorização é aplicada.

Tabela 3.4: Camadas de aplicação do modelo de autorização

Camada	Fonte de decisão	Regra aplicada
Resolução do contexto	Cognito + B0_Users + B0_Roles	O sistema valida o token, confirma o estado da conta, identifica a aplicação do perfil e resolve as permissões efetivas do operador.
Guardas de rota	React Router	<code>ProtectedRoute</code> exige sessão; <code>BackofficeRoute</code> e <code>PanelRoute</code> decidem se o utilizador pode entrar em cada aplicação.
Acesso a páginas	B0_PageGates	Cada página declara as permissões mínimas necessárias; quando há várias permissões, aplica-se lógica OR.
Ações na interface	<code>usePermission</code>	Botões e ações sensíveis só são apresentados quando a permissão necessária existe no contexto do utilizador.
API	<code>requirePermission</code>	O servidor revalida permissões antes de executar a operação, devolve 401/403 quando necessário e regista acessos negados.

Existem também regras de exceção explícitas. O Super Admin contorna todas as validações de página e de endpoint. Os perfis com `roleApplication = backoffice` são tratados como administradores globais: podem entrar no backoffice de gestão e, quando acedem ao painel da JustCauses, contornam os *page gates* e o *middleware* de permissões específico dessa aplicação. Em sentido inverso, um perfil associado apenas a `just_causes` não pode entrar no backoffice administrativo. O perfil `Default`, embora exista como *fallback* interno, não concede acesso útil a nenhuma aplicação.

Esta opção permitiu evitar uma matriz estática demasiado rígida. Em vez de fixar tudo por perfil e por página no código, as permissões disponíveis, os perfis e os *page gates* ficam persistidos no DynamoDB, nas tabelas `B0_Permissions`, `B0_Roles` e `B0_PageGates`. Na prática,

isto tornou o modelo mais fácil de ajustar durante o estágio, sobretudo quando foi necessário corrigir acessos sem reabrir o ciclo de *deploy*.

3.6 Modelo de dados

O modelo de dados do painel reflete a separação arquitetural entre os dados transacionais, persistidos no PostgreSQL, e os dados de auditoria, persistidos no AWS DynamoDB.

No PostgreSQL, as entidades centrais são o User, o Campaign, a Donation, o Withdrawal e o KYC Document, todas com chave primária em UUID e campos de marca temporal de criação e atualização. As relações de integridade referencial seguem o modelo relacional: uma campanha referencia o utilizador criador; uma doação referencia uma campanha e um utilizador doador; um levantamento referencia uma campanha; um documento KYC referencia um utilizador com perfil de criador. As decisões de moderação (aprovação ou rejeição de campanhas e documentos KYC) são também persistidas como registos associados à entidade alvo, com o identificador do operador responsável.

No DynamoDB, residem quatro tipos de dados do painel: os registos de auditoria (BO_Logs), os perfis de acesso (BO_Roles), as permissões disponíveis (BO_Permissions) e a configuração de acesso por rota (BO_PageGates). A tabela de auditoria segue um modelo de chave composta: a chave de partição é o identificador da entidade afetada, e a chave de ordenação é a marca temporal em formato ISO 8601, garantindo consultas eficientes por entidade e por intervalo de tempo. Os registos são imutáveis e o TTL nativo do DynamoDB gere a expiração automática de entradas antigas. A opção por armazenar roles, permissões e *page gates* no DynamoDB, em vez de no PostgreSQL, permite atualizar essas configurações em tempo de execução, sem necessidade de migração de esquema ou de nova publicação da aplicação.

3.7 Diagramas de sequência do sistema

Os diagramas de sequência do sistema (System Sequence Diagram (SSD)) descrevem as interações entre os atores e o sistema para os fluxos mais representativos do painel. Os diagramas apresentados nesta secção são simplificados, com dois participantes (ator e sistema), e focam-se nos cenários principais e nas alternativas de maior relevância. Os diagramas detalhados, com a decomposição interna em camadas de *front-end* e *back-end*, encontram-se no Apêndice A.

Nesta versão do relatório, não se apresentam diagramas dedicados aos fluxos de gestão de utilizadores, fila de KYC, *reports* e gestão de funções. Esses módulos continuam a existir no produto, mas o objetivo aqui é concentrar os SSDs nos fluxos que melhor mostram a estrutura transversal do sistema e os módulos operacionais que ficaram mais maduros durante o estágio.

3.7.1 Resolução de permissões e acesso a páginas

Embora este fluxo não esteja representado com um diagrama autónomo nesta versão reduzida, a sequência real é estável e repete-se em todo o sistema:

1. O operador autentica-se e o *front-end* obtém a sessão Cognito.
2. O *back-end* resolve o utilizador interno, a aplicação associada ao perfil e as permissões efetivas.

3. O *router* e os *page gates* decidem se a aplicação e a página podem ser abertas.
4. Antes de qualquer operação sensível, o *middleware* da API volta a validar as permissões e regista recusas quando necessário.

3.7.2 Autenticação (BOSH-26)

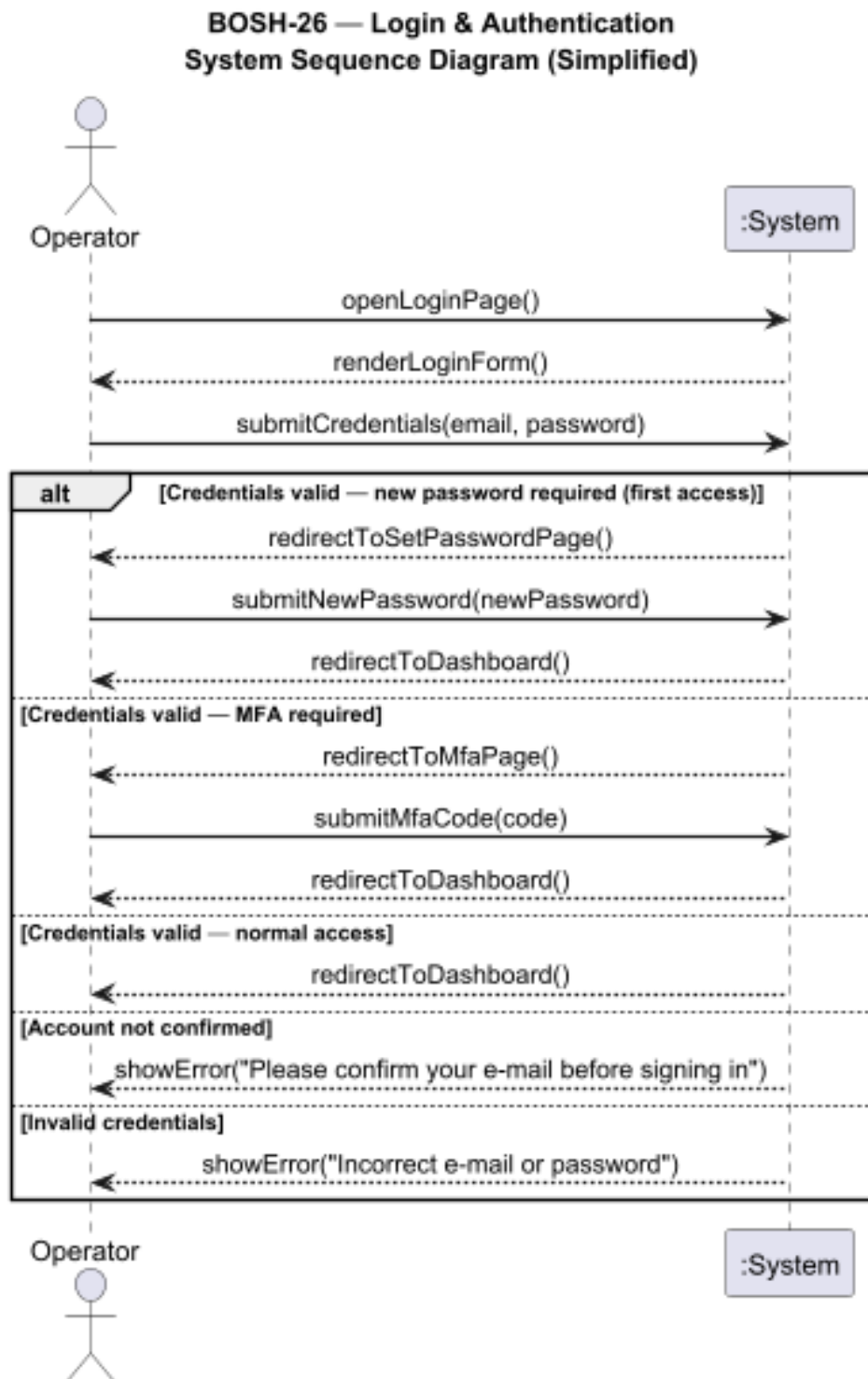


Figura 3.4: SSD — Autenticação (BOSH-26)

3.7.3 Revisão de campanhas (BOSH-1)

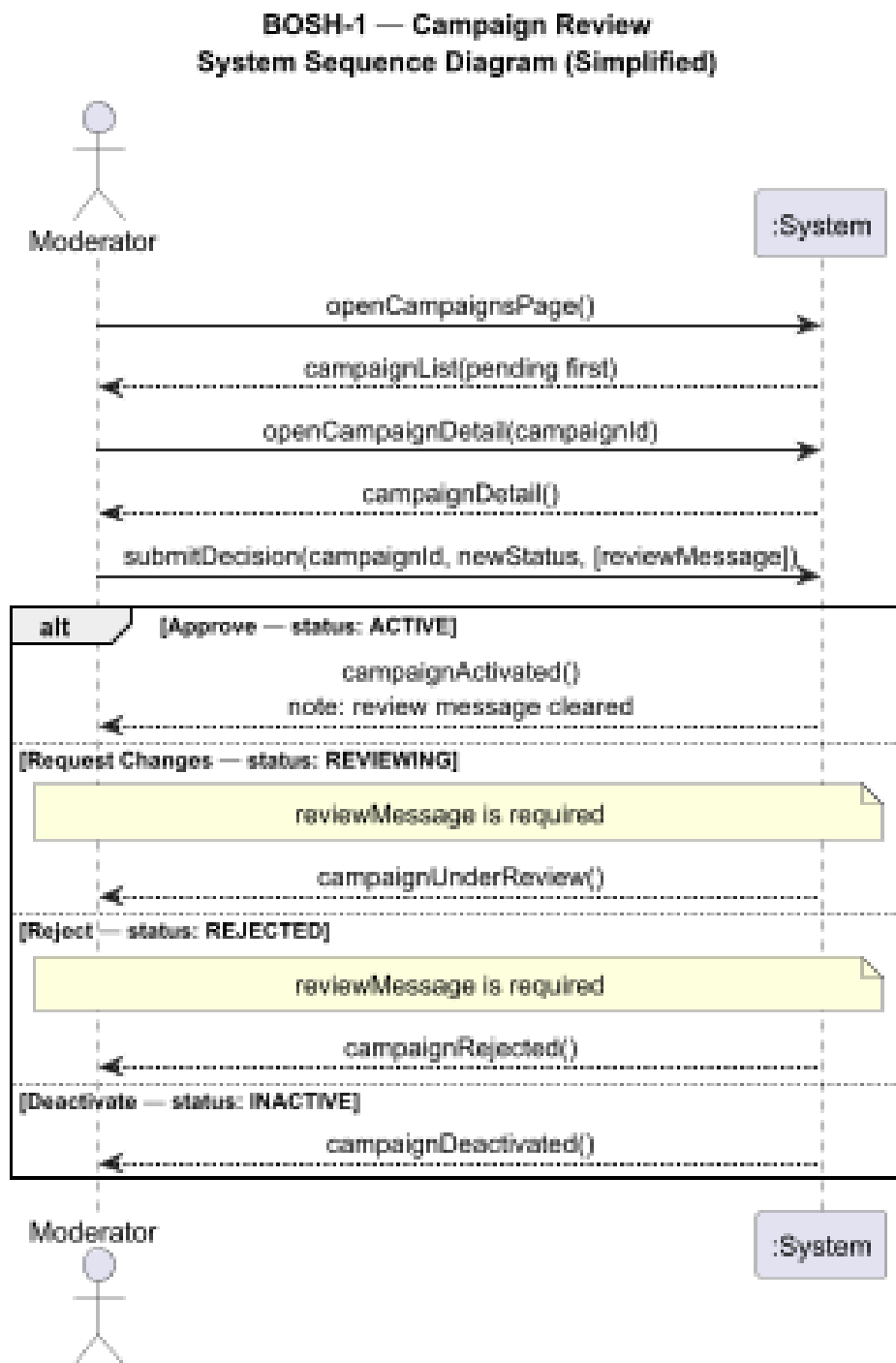


Figura 3.5: SSD — Revisão de campanhas (BOSH-1)

3.7.4 Gestão de categorias (BOSH-7)

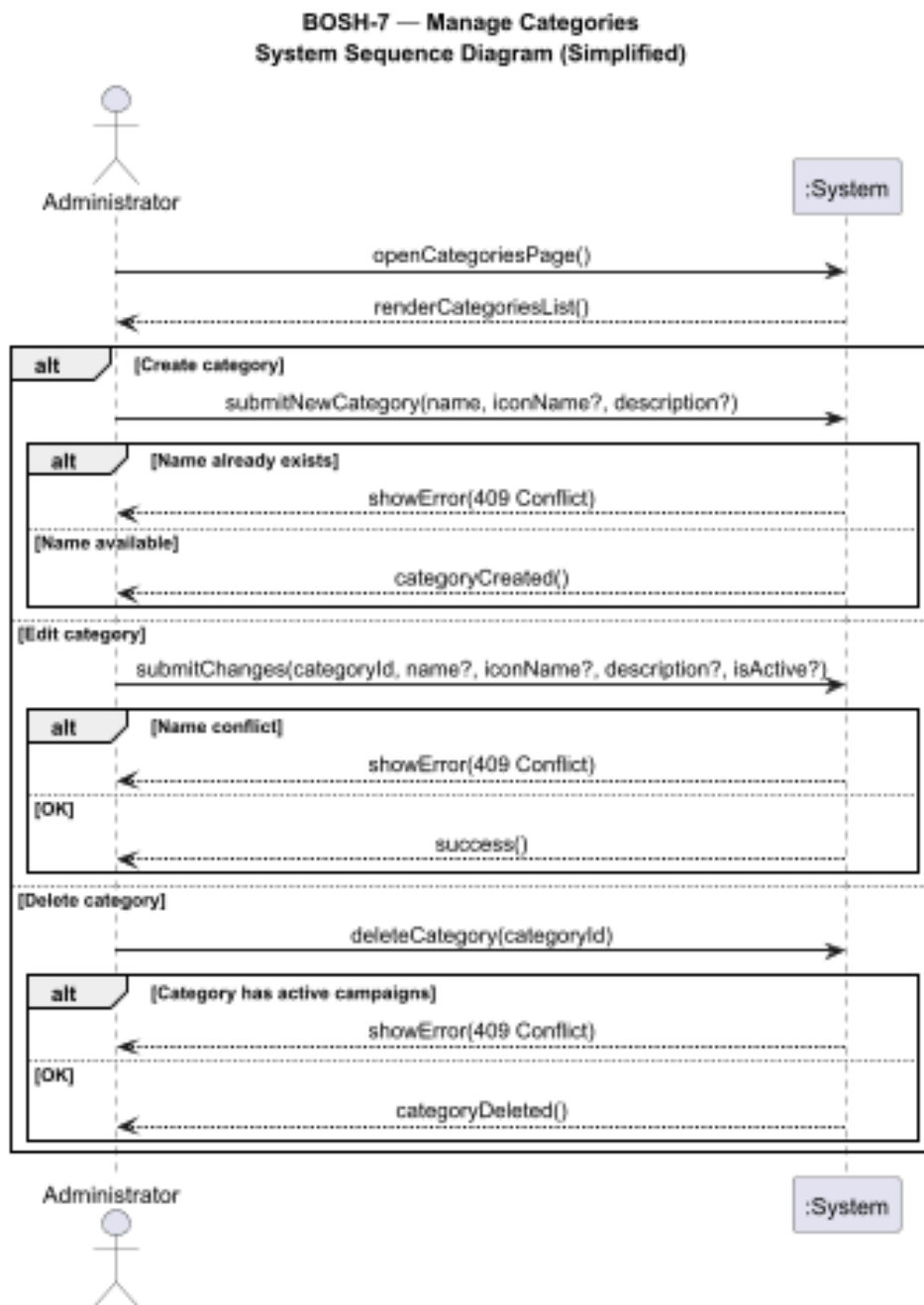


Figura 3.6: SSD — Gestão de categorias (BOSH-7)

3.7.5 Aprovação de levantamentos (BOSH-5)

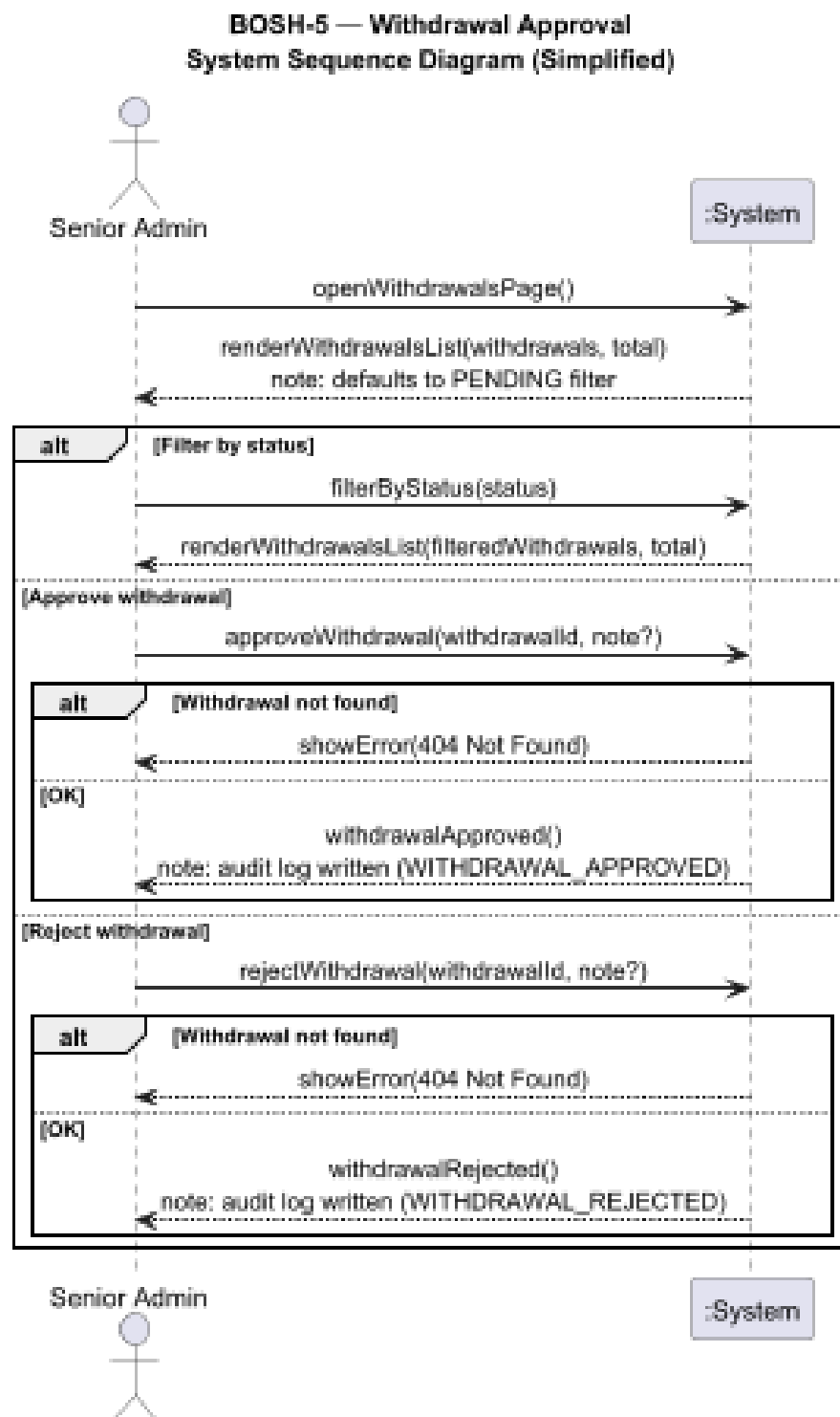


Figura 3.7: SSD — Aprovação de levantamentos (BOSH-5)

3.8 Alternativas ao design

Ao longo das fases de análise e de implementação, foram identificadas alternativas às decisões de desenho tomadas. Esta secção regista as principais opções consideradas e as razões que levaram às escolhas finais.

Framework de administração vs. implementação customizada. O React Admin e o AdminJS foram considerados como ponto de partida para o *front-end* do painel, dado o conjunto de componentes prontos que oferecem para listagens, formulários e autenticação. No entanto, o repositório da empresa já continha uma base de código React com componentes próprios adaptados ao *design system* da *OnlyHighIQ* e à estrutura das APIs existentes. A adoção de uma *framework* de administração teria implicado uma refatoração significativa dessa base, introduzindo uma dependência estrutural de longo prazo e potencialmente limitando a flexibilidade de personalização. Optou-se pela implementação customizada, que mantém total controlo sobre os componentes e a arquitetura da interface.

Verificação de permissões centralizada vs. verificação por módulo no *front-end*. Uma alternativa considerada foi delegar inteiramente ao *front-end* a verificação de permissões, ocultando módulos inacessíveis na interface. Esta abordagem, porém, não constitui uma barreira de segurança: um utilizador com acesso ao token JWT poderia invocar diretamente os endpoints da API sem passar pela interface. A solução adotada aplica o *middleware* `requirePermission` a cada rota Only Just Causes (OJC) no *back-end*, garantindo que a autorização é verificada ao nível da API, independentemente do cliente. O *front-end* complementa esta proteção com *page gates* que ocultam módulos inacessíveis na interface, melhorando a usabilidade sem comprometer a segurança.

Persistência imediata vs. padrão *draft+save* na configuração do acesso. Na configuração administrativa do acesso, uma alternativa seria persistir cada alteração de *toggle* imediatamente, sem exigir confirmação explícita. Embora mais imediata, esta abordagem introduz o risco de alterações parciais — um operador que abandone a página a meio da configuração deixa o sistema num estado inconsistente. O padrão *draft+save* adotado — com estado de rascunho local por cartão de perfil e botões de **Guardar/Descartar** — garante que apenas configurações completas e intencionais são persistidas, eliminando este risco sem impacto significativo na usabilidade.

Auditoria em PostgreSQL vs. DynamoDB. A alternativa de persistir os registos de auditoria numa tabela dedicada do PostgreSQL foi considerada pela sua simplicidade de configuração. No entanto, misturar dados de auditoria de alta frequência de escrita com dados transacionais na mesma base de dados introduz pressão sobre o *connection pool* e pode degradar o desempenho das operações críticas. O **DynamoDB** foi selecionado pela sua escalabilidade horizontal nativa, latência de escrita consistentemente baixa e suporte a Time To Live (TTL) automático para gestão do ciclo de vida dos registos, sem impacto na base de dados relacional principal.

Arquitetura de aplicação única vs. seletor multi-aplicação. Poderia ter-se desenvolvido um painel de administração exclusivo para a JustCauses, sem suporte a múltiplas aplicações.

No entanto, a especificação interna da empresa prevê a incorporação futura de outras plataformas do ecossistema no mesmo sistema de administração. A arquitetura de seletor multi-aplicação adotada — com separação clara entre módulos transversais (autenticação, gestão do acesso, auditoria) e módulos específicos de cada aplicação — permite esta extensão sem redesenho da base, ao custo de uma complexidade ligeiramente superior no modelo de permissões e no roteamento.

3.9 Conclusão

Este capítulo descreveu o domínio do problema, os requisitos, a arquitetura e o desenho do painel de administração da JustCauses. Em vez de prolongar o relatório com descrições muito específicas de páginas isoladas, concentrou-se no que sustenta o comportamento do sistema: a separação entre aplicações, a resolução de permissões, os *page gates* e a revalidação no *back-end*. Os requisitos funcionais foram organizados em treze capacidades centrais, com destaque para autenticação, autorização, operação da plataforma e rastreabilidade. A arquitetura adotada, descrita com recurso ao modelo C4, separa claramente o *front-end* React do *back-end* Node.js, com persistência em PostgreSQL e DynamoDB e infraestrutura inteiramente na AWS. O modelo de permissões deixa de ser visto apenas como uma grelha de perfis e passa a ser entendido como um mecanismo transversal que decide, em cada pedido, o âmbito de acesso do operador. O capítulo seguinte descreve como esse desenho foi concretizado no código e na interface.

Capítulo 4

Implementação da Solução

Neste capítulo descreve-se a implementação do painel de administração da JustCauses, apresentando as soluções técnicas adotadas em cada módulo e evidências do trabalho desenvolvido. A estrutura do capítulo acompanha a ordem de desenvolvimento: começa-se pela infraestrutura base e pelos módulos transversais ao ecossistema (autenticação, controlo de acesso e auditoria), e depois descrevem-se os módulos operacionais da JustCauses por ordem de sprint. Cada secção inclui exemplos representativos de código ou de interface e os resultados dos testes associados.

4.1 Ambiente e estrutura do projeto

O repositório está organizado em duas pastas raiz independentes: *be/* para o *back-end* e *fe/* para o *front-end*, cada uma com a sua configuração de dependências e de compilação. O *back-end* arranca com *ts-node* em modo de desenvolvimento e compila para JavaScript em produção; o *front-end* usa o servidor de desenvolvimento do Vite com *Hot Module Replacement*. O controlo de versões segue uma estratégia de *branching* com ramos de funcionalidade (*feature/**) e revisão de código por *pull request* antes da integração no ramo principal.

A injeção de dependências no *back-end* é configurada no ficheiro *be/src/loaders/index.ts*, que regista todos os repositórios, serviços e controladores no contentor do TypeDI antes de o Express montar as rotas. Esta separação garante que as dependências são resolvidas em tempo de arranque e não em tempo de pedido, evitando instanciações desnecessárias.

A entrega para o ambiente de desenvolvimento está parcialmente automatizada com Bitbucket Pipelines, em fluxos separados para o *front-end* e para o *back-end*. No *front-end*, cada atualização ao ramo *developer* instala as dependências, executa a *build* com Vite, publica a pasta *dist/* num *bucket* Amazon S3 dedicado ao ambiente de desenvolvimento e invalida a distribuição CloudFront para garantir a propagação imediata dos novos *assets*. No *back-end*, o pipeline constrói uma imagem Docker da aplicação, autentica-se no Amazon ECR através da AWS CLI, aplica uma etiqueta imutável baseada em *timestamp* UTC e publica tanto essa versão como a etiqueta *latest*. Esta separação revelou-se útil durante o estágio, porque permitiu iterar a interface e a API com ritmos diferentes sem introduzir acoplamento desnecessário no processo de entrega.

4.2 Autenticação e controlo de acesso

A autenticação é gerida pelo AWS Cognito com o fluxo OAuth 2.0 / JWT padrão. No entanto, a parte mais importante da implementação não é o *login* em si. O que realmente estrutura o painel é a sequência de decisões que acontece depois: perceber quem é o operador, a

que aplicação pode aceder, que páginas pode abrir e que pedidos o *back-end* deve aceitar ou rejeitar. Para tornar este mecanismo mais claro, a sua implementação é descrita aqui por passos.

4.2.1 Passo 1 — autenticar e resolver o contexto do operador

No *front-end*, a sessão é gerida pelo AWS Amplify. Depois de uma autenticação bem-sucedida, o AuthProvider não fica apenas com os dados básicos devolvidos pelo Cognito. Faz também um pedido ao endpoint `/users/me` para obter o contexto interno do operador: identificador no backoffice, perfil atribuído, aplicação associada ao perfil, indicador de perfil *default* e lista de permissões. Esta segunda leitura é essencial, porque o token Cognito identifica a sessão, mas não descreve sozinho as regras de autorização do painel.

No *back-end*, esse contexto é montado no *middleware* `cognitoAuth.ts`. O token Bearer é validado contra o *JWKS* público do Cognito, o `token_use` tem de ser `access`, o `client_id` tem de coincidir com a configuração da aplicação e o sub do Cognito é usado para localizar o operador na base de dados interna. Só depois disso o sistema resolve o perfil em DynamoDB, extrai as permissões efetivas, determina o `roleApplication` e anexa tudo a `req.auth`. Se a conta não existir no backoffice ou estiver desativada, o pedido é rejeitado logo neste ponto.

Há apenas três exceções públicas a esta regra global: `/status`, `/auth/login-failed` e `/logs/access-attempt`. Mesmo nesses casos, a implementação tenta anexar autenticação opcional quando existe um token, sem bloquear o pedido se ele faltar ou for inválido.

4.2.2 Passo 2 — decidir o acesso à aplicação e à página

O roteamento protegido do *front-end* está dividido em quatro guardas: `ProtectedRoute`, `PublicRoute`, `BackofficeRoute` e `PanelRoute`. A distinção importante está nas duas últimas. A `BackofficeRoute` só permite a entrada a perfis associados à aplicação backoffice, enquanto a `PanelRoute` aceita perfis `just-causes` e também perfis backoffice, que funcionam como administradores globais. Quando a aplicação é recusada, a interface não falha em silêncio: mostra um ecrã de acesso negado e envia, em segundo plano, um registo de tentativa falhada.

Depois de entrar na aplicação certa, o sistema ainda tem de decidir a que página encaminhar o operador. Essa decisão é feita duas vezes. Primeiro, no *router*, através das listas `PANEL_PAGES` e `ADMIN_PAGES`, que definem a permissão base de cada rota e calculam o primeiro destino acessível. Depois, já dentro dos *layouts*, através dos *page gates* carregados da API. No painel `JustCauses`, o *layout* aplica lógica OR às permissões configuradas e concede *bypass* a perfis globais de backoffice. No backoffice administrativo, o *bypass* fica reservado ao Super Admin.

Esta combinação revelou-se especialmente útil porque separa duas decisões diferentes. As guardas de rota decidem se faz sentido entrar numa aplicação. Os *page gates* decidem se uma página concreta deve ou não aparecer na navegação e ser renderizada.

4.2.3 Passo 3 — limitar ações dentro da interface

Mesmo quando a página já está aberta, nem todas as ações são mostradas a todos os operadores. Para isso existe o *hook* `usePermission`, usado em páginas e componentes para esconder botões, ações rápidas e controlos de edição. A regra é simples: o Super Admin passa sempre; nos restantes casos, a ação só aparece se a permissão necessária estiver presente no contexto do utilizador.

Este detalhe merece ser sublinhado. No painel, esconder uma ação é uma decisão de experiência de utilização, não uma garantia de segurança. Serve para reduzir ruído visual, evitar cliques inúteis e tornar mais clara a área de responsabilidade de cada operador. A barreira real continua a estar no servidor.

4.2.4 Passo 4 — revalidar cada pedido no servidor

Todas as rotas da API são montadas sob `requireCognitoAuth`, definido no carregador do Express. Em cima dessa autenticação base, os endpoints sensíveis usam o `middleware requirePermission`. O padrão repete-se de forma consistente ao longo do código: GET `/api/ojc/campaigns` exige `view_campaigns`, a criação ou edição de categorias recorre a `change_settings` e a consulta de `/logs` aceita `view_audit_logs` ou `view_admin_logs`. Nas restantes rotas operacionais, a lógica mantém-se: o pedido só avança quando traz uma das permissões aceites para essa operação.

O comportamento do `middleware` é deliberadamente conservador. Se não existir contexto de autenticação, o pedido recebe 401. Se o utilizador estiver autenticado mas não tiver nenhuma das permissões exigidas, recebe 403. O Super Admin contorna sempre esta validação. Além disso, perfis com `roleApplication = backoffice` também passam na verificação dos endpoints, porque no desenho do sistema são tratados como administradores globais.

Quando um pedido é recusado por falta de permissões, o servidor infere a aplicação a partir da rota, regista o evento `APP_ACCESS_FAILED` e aplica uma janela curta de deduplicação para não encher o histórico com erros repetidos do mesmo utilizador sobre o mesmo recurso. Foi uma decisão pequena, mas muito útil: sem ela, bastava atualizar uma página proibida várias vezes para tornar a auditoria difícil de ler.

O excerto da Listagem 4.1 mostra o núcleo desta segunda validação no servidor.

```
1 export function requirePermission(...required) {
2   return (req, res, next) => {
3     const auth = req.auth;
4     if (!auth) {
5       res.status(401).json({ message: "Unauthorized" });
6       return;
7     }
8
9     if (auth.isSuperAdmin || auth.roleApplication === "backoffice") {
10      next();
11      return;
12    }
13
14    const hasPermission = required.some((p) =>
15      auth.permissions.includes(p),
16    );
17
18    if (!hasPermission) {
19      res.status(403).json({
20        message: "Forbidden: insufficient permissions",
21      });
22      return;
23    }
24
25    next();
26  };
27 }
```

Listing 4.1: Validação de permissões no middleware `requirePermission`

4.3 Módulos transversais ao ecossistema

4.3.1 Seletor de aplicações

Após a autenticação, o utilizador é redirecionado para o seletor de aplicações (`/backoffice/panel`), implementado na página `AdminOverviewPage`. Esta página apresenta a lista de aplicações do ecossistema OnlyHighIQ às quais o operador tem acesso, com o nome da aplicação, o perfil atribuído e a data do último acesso. A seleção de uma aplicação redireciona para o painel específico dessa aplicação. Caso o operador não tenha acesso a nenhuma aplicação, é apresentada uma mensagem de acesso negado.

A navegação entre aplicações é feita sem necessidade de nova autenticação — a sessão Cognito é partilhada entre todas as aplicações do ecossistema. O cabeçalho do painel exhibe permanentemente o nome da aplicação corrente, o nome e o perfil do operador, e um botão de regresso ao seletor.

4.3.2 Configuração administrativa do acesso

A área `/admin` concentra as ferramentas administrativas que mantêm o modelo de autorização vivo. Nela existem páginas para gerir contas internas, ajustar perfis e, sobretudo, configurar os *page gates* das duas aplicações. Neste relatório, estas páginas não são tratadas como módulos funcionais isolados; são apresentadas como a consola onde o mecanismo de autorização é mantido e afinado.

A `PageGatesPage` carrega as configurações por aplicação e permite associar a cada página um conjunto de permissões. A regra usada tanto na interface como no *back-end* é deliberadamente simples: se uma página lista várias permissões, basta o operador ter uma delas. Isto permitiu adaptar o acesso sem mexer no código das rotas e sem novo *deploy*, porque a configuração fica persistida em `DynamoDB`.

Também aqui foi mantido o padrão *draft*. O operador pode experimentar alterações, compará-las com o estado atual e só depois decidir guardar ou descartar. Num módulo tão sensível como o de autorização, este detalhe fez diferença, porque evita mudanças acidentais aplicadas à primeira interação.

4.3.3 Sistema de auditoria

Todas as operações de escrita realizadas por operadores sobre entidades críticas geram um registo de auditoria no `AWS DynamoDB`. Cada registo contém o identificador do ator, o tipo e identificador da entidade afetada, a ação executada, a marca temporal em ISO 8601 e metadados adicionais específicos da ação (razão, estado anterior, estado novo). Os registos são imutáveis — não existem operações de atualização ou eliminação no `AuditLogRepo`.

A página `LogsPage` permite consultar o histórico de auditoria com filtros por data, por operador e por tipo de ação. Para além das ações de negócio, o sistema passou também a registar tentativas falhadas de acesso à aplicação e recusas por permissões insuficientes. No *front-end*, esse envio é feito de forma não bloqueante; no *back-end*, a recusa é registada no próprio *middleware* de autorização. A chave de partição no `DynamoDB` é o identificador da entidade,

com a marca temporal como chave de ordenação, o que garante uma consulta eficiente por entidade e por intervalo de tempo.

4.4 Módulos operacionais da JustCauses

Os módulos descritos nesta secção são os que melhor mostram a integração com os dados transacionais da plataforma e a consistência da arquitetura adotada. Outros fluxos existentes no produto, como consulta de utilizadores, fila de verificação e tratamento de *reports*, são aqui apenas assumidos como parte do universo operacional, sem desenvolvimento próprio no texto, para não desviar o foco do mecanismo transversal de autorização.

4.4.1 Dashboard e métricas

O painel de operações da JustCauses é composto por duas páginas distintas com propósitos diferentes. A *DashboardPage* funciona como seletor de aplicações do ecossistema: apresenta as aplicações a que o operador tem acesso com base no seu perfil, mostra o seu papel (*role*) em cada aplicação e a última data de acesso (lida a partir do *localStorage* sob a chave *bo_last_accessed*). A *AdminOverviewPage*, disponível em */ojc* após a seleção da aplicação JustCauses, é a visão operacional da plataforma, com sete métricas de resumo calculadas pelo endpoint *GET /api/ojc/overview/stats*: número total de utilizadores, campanhas ativas, volume total angariado, pedidos de aprovação pendentes, denúncias abertas, total de campanhas e doações concluídas.

O *back-end* calcula estas métricas através de *queries* de agregação SQL sobre a base de dados da JustCauses. Os resultados são devolvidos num único pedido e apresentados em cartões de resumo no topo da página.

4.4.2 Gestão de campanhas

O módulo de gestão de campanhas, acessível na página *CampaignsPage* em */ojc/campaigns*, suporta o fluxo completo de revisão de campanhas submetidas. O *back-end* expõe três endpoints: *GET /api/ojc/campaigns* para a listagem com filtro por estado, *GET /api/ojc/campaigns/:id* para o detalhe de uma campanha, e *PATCH /api/ojc/campaigns/:id/status* para a atualização do estado.

A fila de pendentes é a vista padrão (filtro *PENDING*), podendo o operador filtrar por qualquer estado: *PENDING*, *ACTIVE*, *INACTIVE*, *FINISHED*, *REJECTED* e *REVIEWING*. O fluxo de revisão suporta três ações: **aprovar** (transita para *ACTIVE*), **rejeitar** com razão obrigatória (transita para *REJECTED*) e **pedir alterações** com mensagem obrigatória (transita para *REVIEWING*). A mensagem de revisão (*reviewMessage*) é persistida na base de dados e apresentada na listagem, permitindo ao operador acompanhar o estado das campanhas em processo de correção. O estado *REVIEWING* está também disponível para campanhas já *ACTIVE*, suportando revisões pós-publicação.

Cada ação de revisão gera automaticamente: um registo de auditoria no DynamoDB, uma notificação por e-mail ao criador via Simple Email Service (SES), e a exibição de um *badge* de verificação do criador na interface, caso o mesmo tenha o processo KYC concluído.

4.4.3 Gestão de categorias

O módulo de categorias, acessível na página `CategoriesPage` em `/ojc/categories`, implementa a gestão completa das categorias temáticas das campanhas. Cada categoria é definida por um nome único, um *slug* de URL, um ícone (*iconName*), uma descrição e um estado de ativação (*isActive*).

O *back-end* expõe quatro endpoints: `GET /api/ojc/categories` para listagem, `POST /api/ojc/categories` para criação, `PATCH /api/ojc/categories/:id` para edição (incluindo desativação via *isActive*: `false`) e `DELETE /api/ojc/categories/:id` para remoção. A criação e a edição validam a unicidade do nome e do *slug*, devolvendo `409 Conflict` em caso de duplicado. Todas as operações de escrita são registadas no sistema de auditoria, com a ação `CATEGORY_DELETED` para eliminações.

4.4.4 Analytics

O módulo de *analytics*, acessível na página `AnalyticsPage` em `/ojc/analytics`, apresenta uma visão temporal e categórica do desempenho da plataforma. O *back-end* expõe o endpoint `GET /api/ojc/analytics` com suporte a filtros de período (`1d`, `1w`, `2w`, `1m`, `3m`, `6m`, `1y`, `all`).

A página inclui gráficos de evolução temporal de receita, novos utilizadores e campanhas criadas; uma tabela de campanhas com mais doações angariadas, ordenada por valor total decrescente (`ORDER BY COALESCE(SUM(d."amount"), 0) DESC`); e distribuições de doações por estado e por categoria. A *query* de agregação das campanhas utiliza `COALESCE` para garantir que campanhas sem doações surgem corretamente com valor zero em vez de serem excluídas.

4.4.5 Aprovação de levantamentos

O módulo de levantamentos, acessível na página `WithdrawalsPage` em `/ojc/withdrawals`, permite acompanhar os pedidos de transferência de fundos submetidos pelos criadores. A listagem apresenta o e-mail do criador, o endereço de carteira (abreviado), o montante em criptomoeda e o equivalente em *fiat*, o fornecedor e método de pagamento, a data de submissão, o estado e o *hash* da transação com ligação ao `BaseScan` para verificação na *blockchain*.

O *back-end* expõe dois endpoints: `GET /api/ojc/withdrawals` com filtro por estado (`PENDING`, `COMPLETED`, `FAILED`) e paginação, e `PATCH /api/ojc/withdrawals/:id/status` para transitar o estado para `COMPLETED` ou `FAILED`. A *query* de listagem realiza um `JOIN` entre as tabelas `Withdrawals`, `Wallet` e `Account` para obter o e-mail do criador associado a cada carteira. Cada atualização de estado regista um evento de auditoria no `DynamoDB` com a ação `WITHDRAWAL_APPROVED` ou `WITHDRAWAL_REJECTED` e uma nota opcional do operador.

4.4.6 Transações

O módulo de transações, acessível na página `TransactionsPage` em `/ojc/transactions`, permite visualizar o histórico de transações financeiras da plataforma, incluindo doações em `USDC` na rede `Base`. A listagem apresenta informação sobre o doador, o criador destinatário, o montante, o estado da transação e o identificador da campanha associada.

4.5 Testes

A estratégia de testes adotada ao longo do estágio combinou validação manual sistemática com testes automatizados já existentes no ecossistema JustCauses, complementada por um conjunto unitário de testes ao módulo de eventos de domínio do *back-end*.

4.5.1 Validação manual por módulo

A principal abordagem de validação durante o desenvolvimento foi a verificação manual de cada módulo após a sua implementação. Para cada *story* concluída, o ciclo de validação seguiu os seguintes passos: (1) verificação dos cenários de sucesso descritos nos critérios de aceitação do Jira; (2) verificação dos cenários de erro (entradas inválidas, estados inesperados, ausência de permissões); e (3) confirmação da persistência correta dos dados e dos registos de auditoria no DynamoDB.

Esta abordagem foi aplicada sobretudo aos fluxos transversais de autenticação, autorização e auditoria, bem como aos módulos operacionais prioritários de campanhas, categorias, *analytics*, transações e levantamentos. Revelou-se suficientemente ágil para o ritmo de *sprints* semanais. Os resultados de cada validação foram documentados como comentários nas *issues* do Jira antes de mover a tarefa para o estado *Done*.

4.5.2 Testes de integração da API da JustCauses

O repositório contém uma suite de testes de integração de API desenvolvida com o *framework* Playwright, localizada em `JustCauses/tests/tests/api/`. Esta suite cobre os principais módulos da API da JustCauses — campanhas, doações, mensagens, perfis, atualizações, seguimentos e transações — testando os endpoints diretamente com pedidos HTTP autenticados e validando as respostas contra contratos esperados.

Os testes de integração são organizados por recurso, cada ficheiro cobrindo o ciclo completo de operações CRUD do respetivo endpoint. A Tabela 4.1 resume os módulos cobertos.

Tabela 4.1: Módulos cobertos pela suite de testes de integração da JustCauses API

Ficheiro de testes	Módulo coberto
<code>campaigns.spec.ts</code>	Criação, listagem, detalhe e eliminação de campanhas
<code>donations.spec.ts</code>	Fluxo de doações e estados de transação
<code>messages.spec.ts</code>	Sistema de mensagens entre utilizadores
<code>profile.spec.ts</code>	Leitura e atualização de perfis
<code>campaign-updates.spec.ts</code>	Publicação e consulta de atualizações de campanha
<code>followers.spec.ts</code>	Seguimento e cessação de seguimento de campanhas
<code>saved-campaigns.spec.ts</code>	Guardar e remover campanhas da lista pessoal
<code>tips.spec.ts</code>	Envio e consulta de gorjetas
<code>rates.spec.ts</code>	Consulta de taxas de câmbio USDC/fiat
<code>uploads.spec.ts</code>	<i>Upload</i> de ficheiros multimédia

Adicionalmente, a pasta `JustCauses/tests/tests/e2e/` contém testes End to End (E2E) com o Playwright para os fluxos de navegação da plataforma, cobrindo a página inicial, os cartões de campanha e o sistema de mensagens.

4.5.3 Testes unitários ao módulo de eventos de domínio

O *back-end* do painel de administração inclui um conjunto de testes unitários ao módulo `DomainEvents`, localizado em `be/src/core/domain/events/tests/domainEvents.spec.ts`, executados com o Jest. Estes testes verificam o ciclo de vida dos eventos de domínio: subscrição de *handlers*, marcação de agregados para despacho, despacho dos eventos após a marcação e remoção dos agregados da fila após o despacho.

Embora a cobertura de testes automatizados dedicada ao painel de administração seja limitada nesta fase, o módulo de eventos de domínio é um componente transversal crítico, cuja correção é condição necessária para a fiabilidade dos fluxos de notificação e auditoria implementados ao longo do painel.

4.5.4 Limitações e trabalho futuro em testes

A ausência de uma suite de testes de integração dedicada às APIs do painel de administração constitui a principal lacuna de cobertura nesta fase. A razão principal foi a priorização do desenvolvimento funcional dentro dos *sprints* definidos, relegando a automação de testes para uma fase posterior. Como trabalho futuro, prevê-se a adição de testes de integração aos endpoints REST do backoffice com Supertest e Jest, bem como a expansão dos testes E2E ao painel de administração com o Playwright.

4.6 Avaliação da solução

Esta secção avalia o estado de implementação da solução por eixo funcional, com base no estado do painel no final do período de desenvolvimento coberto por este relatório. Em vez de voltar a detalhar cada fluxo operacional isolado, a análise centra-se nas capacidades que melhor representam o trabalho efetivamente consolidado.

Tabela 4.2: Estado de implementação por eixo funcional

Eixo	Descrição resumida	Estado	Observações
Autenticação com Cognito	Gestão de sessão, recuperação de palavra-passe e integração com o painel	Concluído	Fluxo funcional com Amplify no cliente e validação JWT no servidor
Resolução do contexto de autorização	Perfil, aplicação associada e permissões efetivas do operador	Concluído	/users/me no cliente req.auth no servidor mantém a mesma fonte de verdade
Acesso a aplicações e páginas	Guardas de rota, redireções e <i>page gates</i> por aplicação	Concluído	Separação entre backoffice just_causes com lógica OR nas páginas
Proteção de endpoints REST	Verificação de permissões antes de qualquer operação sensível	Concluído	requirePermission aplicado de forma consistente nas rotas críticas
Auditoria técnica e funcional	Registo de ações de escrita e de acessos negados	Concluído	DynamoDB como repositório imutável e deduplicação de recusas repetidas
Dashboard e observabilidade	Visão global da atividade da plataforma	Concluído	AdminOverviewPage AnalyticsPage com métricas agregadas
Revisão de campanhas	Aprovação, rejeição e pedido de alterações	Concluído	Fluxo completo com auditoria e notificação a criador
Gestão de categorias	Criação, edição e desativação de taxonomias	Concluído	CRUD completo com validação de unicidade
Operações financeiras	Consulta de transações e pedidos de levantamento	Parcial	A componente de consulta está pronta; algumas ações de interface ficaram para iteração seguinte
Configuração administrativa do acesso	Manutenção de contas internas, perfis e <i>page gates</i>	Parcial	A base está entregue; outras definições geradas pela plataforma não foram aprofundadas nesta fase
Garantia de qualidade automática	Testes unitários e automação de validação	Parcial	Existe validação manual sistemática, mas a cobertura automatizada ainda é reduzida

O balanço global é positivo. Os eixos mais estruturantes do sistema — autenticação, autorização, controlo de acesso a páginas, proteção de endpoints e auditoria — ficaram concluídos e coerentes entre si. Isto é importante porque, num painel administrativo, a robustez destas camadas pesa mais do que a quantidade de páginas disponíveis.

Os pontos parciais concentram-se sobretudo em duas frentes. A primeira é o fecho de alguns fluxos operacionais na interface, onde o *back-end* já se encontra mais adiantado do que o *front-end*. A segunda é a automação de testes, que ainda não acompanha o mesmo nível de maturidade da arquitetura e da implementação funcional.

4.7 Conclusão

O trabalho de implementação descrito neste capítulo mostra um painel já utilizável em contexto operacional e, ao mesmo tempo, ainda em evolução. O resultado mais sólido não foi apenas a entrega de páginas, mas a consolidação de uma linha técnica clara para autenticação, autorização, auditoria e administração do acesso.

Esta coerência nota-se em vários pontos: o *front-end* e o *back-end* partilham a mesma lógica de permissões, as páginas são filtradas por *page gates* configuráveis, os endpoints são revalidados no servidor e as recusas deixam rasto auditável. Os módulos operacionais entregues assentam todos nesta base comum, o que reduz o custo de evolução nas iterações seguintes. Os pontos que ficaram parciais não decorrem de limitações estruturais da solução. Resultam sobretudo de prioridades de *sprint*, dependências externas e margem de tempo disponível até ao fecho do estágio.

No capítulo seguinte é feita a apreciação global do trabalho realizado, retomando os objetivos inicialmente definidos, as limitações identificadas e as principais linhas de evolução futura da solução.

Capítulo 5

Conclusões

5.1 Objetivos concretizados

Este capítulo avalia o grau de concretização dos objetivos definidos no início do estágio, com base nos resultados efetivamente alcançados. Em vez de voltar a enumerar cada fluxo operacional isolado, a análise centra-se nos objetivos técnicos e funcionais que melhor representam o trabalho efetivamente entregue.

Tabela 5.1: Grau de concretização dos objetivos

Objetivo	Estado	Observações
Autenticação centralizada de operadores	Concluído	Cognito com MFA, gestão de sessão no <i>front-end</i> e validação JWT no <i>back-end</i>
Aplicação de permissões no cliente e no servidor	Concluído	Guardas de rota, <i>page gates</i> , <code>usePermission</code> e <code>requirePermission</code> trabalham sobre o mesmo contexto de acesso
Configuração dinâmica do acesso a páginas	Concluído	<i>Page gates</i> persistidos em DynamoDB e atualizáveis sem novo <i>deploy</i>
Sistema de auditoria e rastreabilidade	Concluído	Registo imutável de ações e de acessos negados, com filtros e histórico consultável
Módulos operacionais prioritários da plataforma	Concluído	Dashboard, campanhas, categorias, transações, <i>analytics</i> e fila de levantamentos disponíveis
Operações financeiras completas	Parcial	A componente de consulta está consolidada; algumas ações de interface ficaram para iteração seguinte
Arquitetura extensível multiaplicação	Concluído	Separação entre backoffice de gestão e painel JustCauses preparada para crescimento do ecossistema
Processo de entrega técnica	Parcial	Há automação de entrega para FE e BE em desenvolvimento, mas ainda sem cobertura alargada de validações automáticas
Garantia de qualidade automatizada	Parcial	Validação manual sistemática; testes automatizados ainda concentrados em áreas específicas do <i>back-end</i>
Interfaces claras para operação diária	Concluído	Navegação coerente, estados de acesso negado explícitos e padrão <i>draft/guardar</i> em áreas sensíveis

O balanço global é positivo. Os objetivos mais estruturantes — autenticação, autorização, controlo de acesso a páginas, auditoria e base arquitetural para crescimento do sistema — ficaram concluídos e coerentes entre si. Os pontos parciais concentram-se sobretudo na automação de qualidade e no fecho de algumas ações de interface, não no desenho de base da solução.

5.2 Limitações e trabalho futuro

5.2.1 Limitações identificadas

As limitações do trabalho realizado distribuem-se por três dimensões: funcionalidades não concluídas, cobertura de testes e integração com módulos externos.

Funcionalidades não concluídas. Alguns fluxos operacionais secundários e algumas ações de interface ficaram fora do período útil do estágio. Em vários casos, o *back-end* chegou primeiro e o fecho da camada de interface ficou para iteração seguinte. A aprovação de levantamentos é o exemplo mais claro desta assimetria: a base de serviço ficou pronta, mas a interação final no *front-end* não foi concluída. Também ficaram de fora funcionalidades dependentes de decisões de produto ou de integrações externas que não estavam maduras durante o estágio.

Cobertura de testes. A ausência de testes de integração automatizados dedicados às APIs do painel de administração constitui uma limitação relevante. A validação foi feita manualmente módulo a módulo, o que é adequado para um MVP mas não escala com o crescimento do sistema. A cobertura de testes unitários está limitada ao módulo de eventos de domínio do *back-end*.

Integração com serviços externos. O sistema de auditoria de acessos (BOSH-30) não confirmou a captura de eventos de *login* bem-sucedido e falhado no fluxo de autenticação do AWS Cognito, pelo que a *AuditTrailPage* filtra apenas por eventos gerados após a autenticação. A integração com o AWS Rekognition para moderação automática de imagens foi identificada como desejável, mas não foi incluída no âmbito deste estágio.

5.2.2 Trabalho futuro

Com base nas limitações identificadas e nas capacidades da arquitetura implementada, propõem-se as seguintes direções de trabalho futuro:

- **Fecho dos fluxos parciais:** concluir as ações de interface ainda em falta nos módulos já servidos pelo *back-end*, com prioridade para operações financeiras e restantes páginas administrativas sensíveis.
- **Reforço do modelo de autorização:** acrescentar mais validação automática aos *page gates*, testes dedicados a permissões e ferramentas de diagnóstico para auditoria de acessos.
- **Extensão do ecossistema:** integrar novas aplicações do ecossistema da *OnlyHighIQ* no mesmo painel de administração, aproveitando a separação já existente entre módulos transversais e módulos específicos de cada aplicação.
- **Notificações em tempo real:** implementar um canal de notificações para operadores baseado em Server-Sent Events (SSE) ou WebSocket, que informe em tempo real sobre eventos operacionais relevantes.

- **Exportação e apoio à auditoria:** acrescentar exportação para Portable Document Format (PDF) e Comma-Separated Values (CSV) nos módulos financeiros e de observabilidade, reforçando o suporte a auditorias internas e externas.
- **Integrações inteligentes:** avaliar a integração do AWS Rekognition e de serviços semelhantes para triagem automática de conteúdo e deteção precoce de padrões anómalos.
- **Testes automatizados:** desenvolver uma suite de testes de integração para as APIs do painel com Jest e Supertest, e expandir os testes E2E com Playwright ao painel de administração.

5.3 Apreciação final

O estágio na *OnlyHighIQ* representou uma experiência formativa de grande densidade técnica e profissional. Ao longo de aproximadamente 14 semanas, foi possível conceber, arquitetar e implementar um sistema de administração funcional para uma plataforma de *crowdfunding* real, com utilizadores, fluxos financeiros e requisitos de conformidade concretos — um contexto muito diferente dos projetos académicos habituais.

Do ponto de vista técnico, o principal desafio foi a complexidade da integração entre múltiplas bases de dados (PostgreSQL partilhado da JustCauses e DynamoDB para auditoria) e a gestão de permissões multi-aplicação com um único sistema de autenticação Cognito. A implementação do *middleware* `requirePermission`, com lógica de *bypass* para Super Admin, deduplicação de eventos negados e registo em DynamoDB, exigiu um entendimento aprofundado do ciclo de pedido HTTP do Express 5 e das garantias de consistência eventual do DynamoDB. A gestão de múltiplos *connection pools* PostgreSQL (partilhado e OJC) com TypeDI foi igualmente um aspeto novo que exigiu estudo e iteração.

Se houve uma parte do estágio que mais me obrigou a sair da lógica confortável de CRUD, foi precisamente a aplicação prática das permissões. À primeira vista parecia apenas uma combinação de *middlewares* e rotas. Na prática, obrigou a alinhar Cognito, dados internos de utilizadores, navegação React, *page gates* configuráveis, *bypasses* legítimos de administração global e registo de tentativas falhadas sem encher a auditoria de ruído. Foi aí que o projeto deixou claramente de parecer um exercício académico e começou a comportar-se como um sistema de produção.

A nível profissional, o trabalho em metodologia SCRUM com *sprints* semanais e gestão de tarefas no Jira trouxe uma disciplina de entrega iterativa que não é fácil de adquirir em contexto académico. A necessidade de comunicar bloqueadores ao supervisor, de estimar esforço de implementação e de documentar as decisões de design em comentários de *issue* foram competências desenvolvidas de forma natural ao longo do estágio.

A experiência na *OnlyHighIQ* foi globalmente positiva. A empresa é uma organização jovem e ambiciosa, com uma cultura de autonomia e responsabilidade individuais que exige maturidade técnica desde o primeiro dia. O facto de trabalhar sobre uma base de código existente — com decisões de arquitetura já tomadas e padrões a respeitar — foi simultaneamente um desafio (compreender o sistema antes de o estender) e uma vantagem (aprender com escolhas já validadas em produção). O resultado final — um painel de administração operacional, com autenticação centralizada, controlo de acesso consistente, auditoria e módulos operacionais prioritários já utilizáveis — é um contributo concreto para a evolução da plataforma JustCauses.

Bibliografia

- [1] OnlyHighIQ, *OnlyHighIQ — Missão, Visão e Valores*, Disponível em <https://www.onlyhighiq.com>, (Último acesso em 31/03/2026), 2025.
- [2] OnlyHighIQ, *Backoffice Only High IQ Ecosystem — Organisation & Full Feature Specification*, Documento interno da empresa, v1.0, 2026.
- [3] S. Sachdeva, «Scrum Methodology,» *International Journal of Engineering and Computer Science*, vol. 5, n.º 6, pp. 16 792–16 799, 2016. doi: 10.18535/ijecs/v5i6.11 URL: <https://doi.org/10.18535/ijecs/v5i6.11>
- [4] Shopify, *Shopify — About*, Disponível em <https://www.shopify.com/about>, (Último acesso em 31/03/2026), 2024.
- [5] Shopify, *Shopify Help Center — Staff accounts and permissions*, Disponível em <https://help.shopify.com/en/manual/your-account/staff-accounts>, (Último acesso em 31/03/2026), 2024.
- [6] Stripe, *Stripe Dashboard*, Disponível em <https://stripe.com/docs/dashboard>, (Último acesso em 31/03/2026), 2024.
- [7] Directus, *Directus — The Open Data Platform*, Disponível em <https://directus.io>, (Último acesso em 31/03/2026), 2024.
- [8] Appsmith, *Appsmith — Open source framework to build internal tools*, Disponível em <https://www.appsmith.com>, (Último acesso em 31/03/2026), 2024.
- [9] S. Brown, *The C4 model for visualising software architecture*, Disponível em <https://c4model.com>, (Último acesso em 31/03/2026), 2018.

Apêndice A

Diagramas de sequência do sistema — Versão detalhada

Este apêndice contém os diagramas de sequência do sistema (SSD) detalhados para os fluxos que ficaram em maior destaque no corpo do relatório. Ao contrário dos diagramas simplificados apresentados na Secção 3.7, estes decompõem as interações internas em camadas: página de *front-end*, *hooks* e serviços React, controlador, serviço e repositório do *back-end*, e bases de dados.

A.1 Autenticação (BOSH-26)

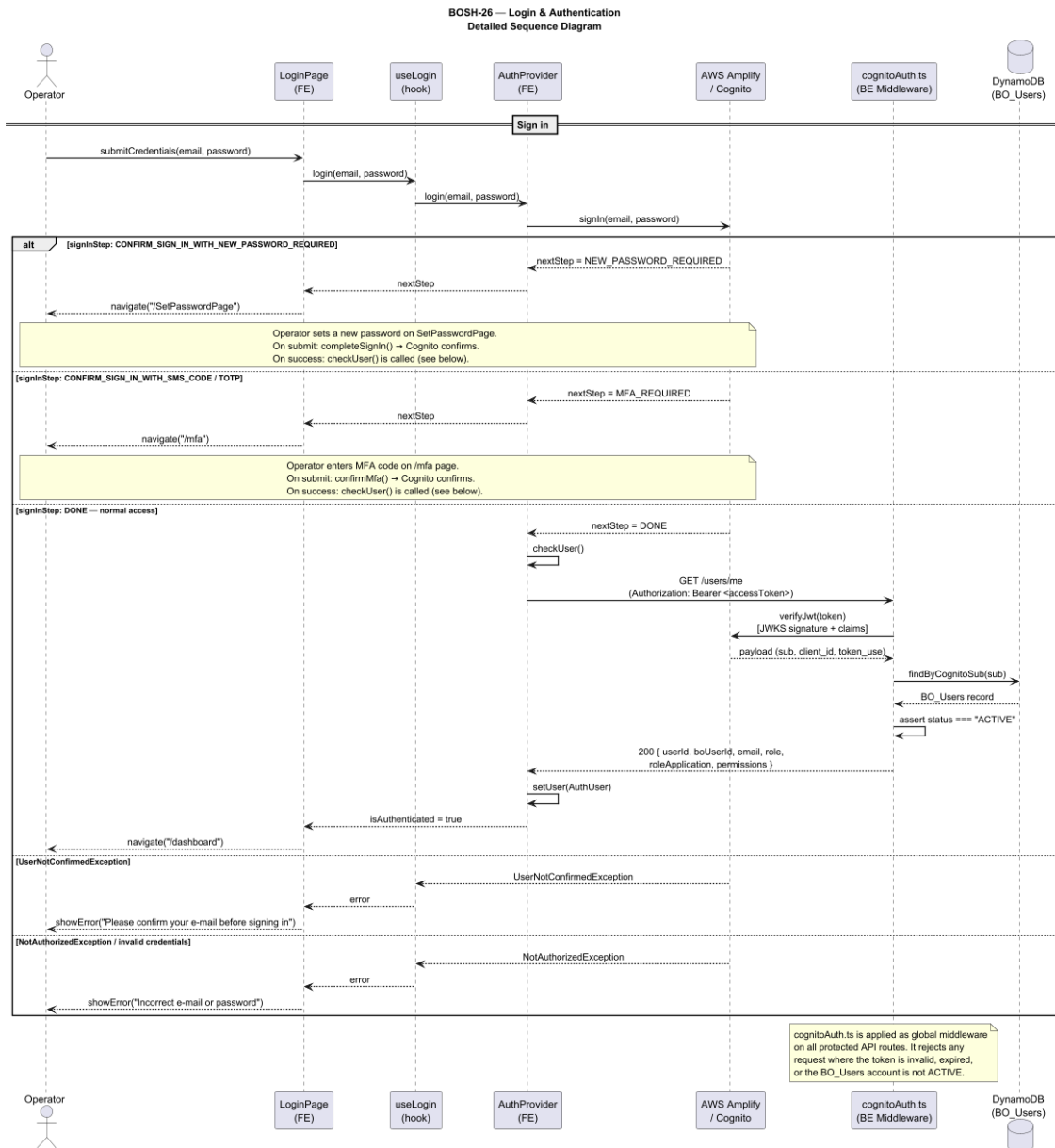


Figura A.1: SSD detalhado — Autenticação (BOSH-26)

A.2 Revisão de campanhas (BOSH-1)

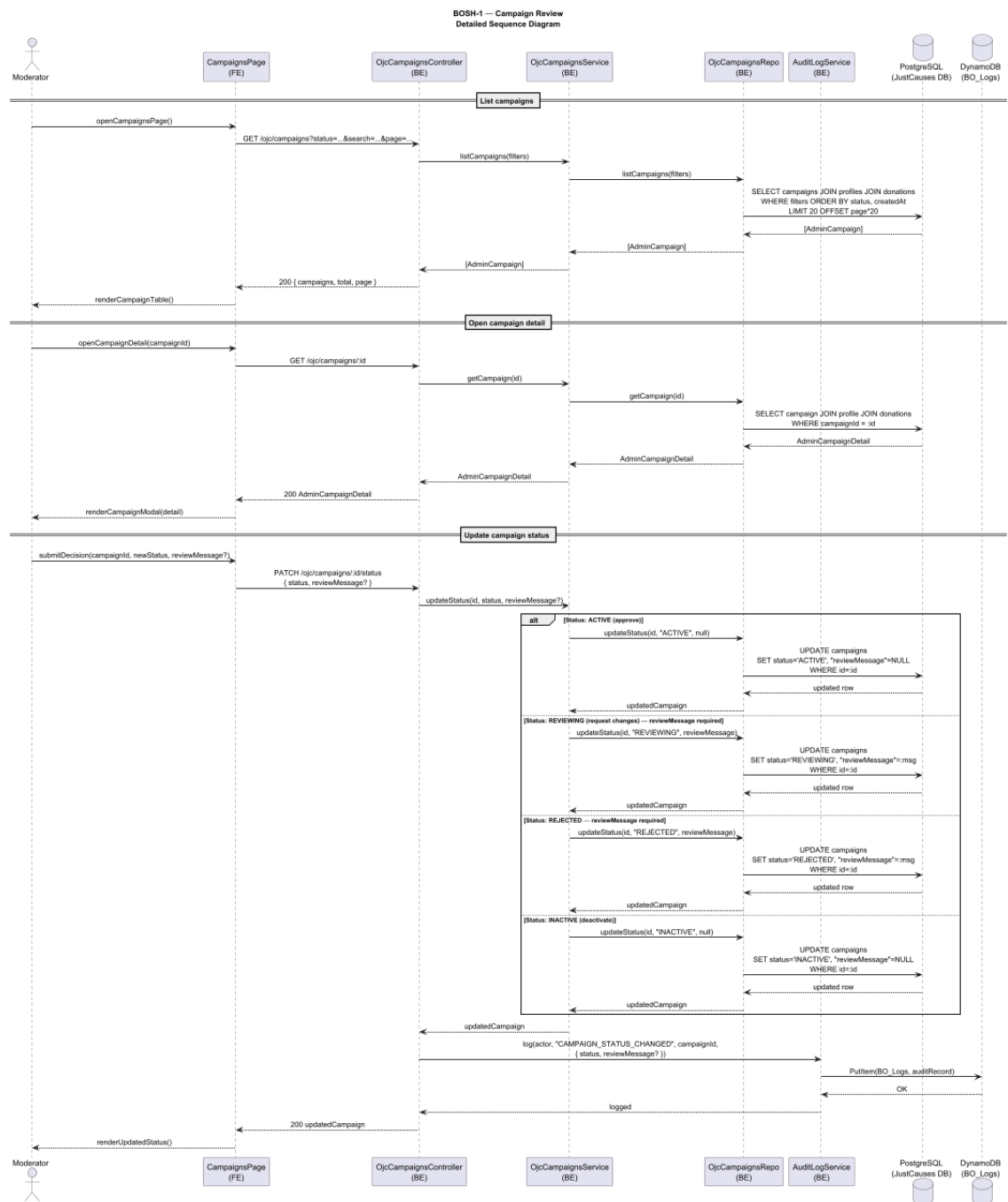


Figura A.2: SSD detalhado — Revisão de campanhas (BOSH-1)

A.3 Gestão de categorias (BOSH-7)

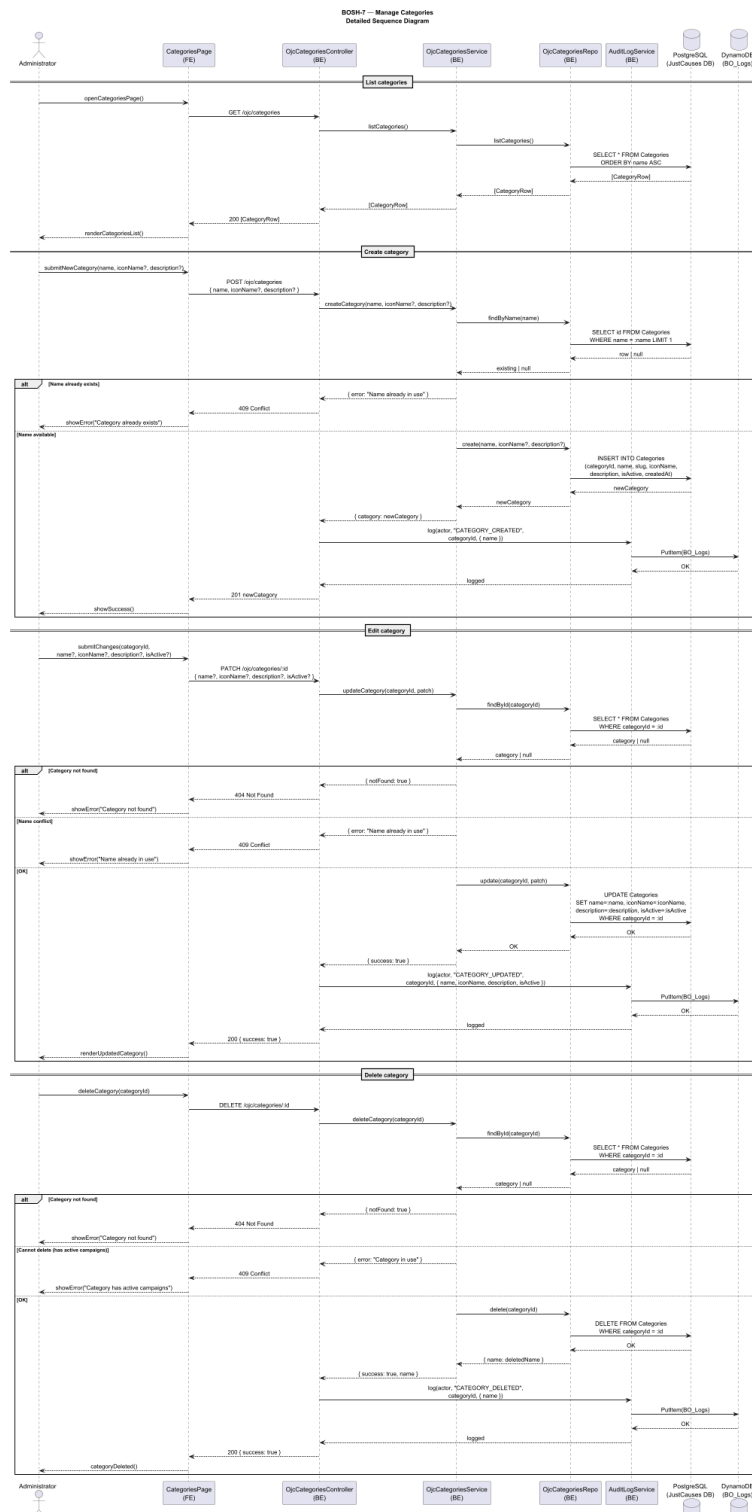


Figura A.3: SSD detalhado — Gestão de categorias (BOSH-7)

A.4 Aprovação de levantamentos (BOSH-5)

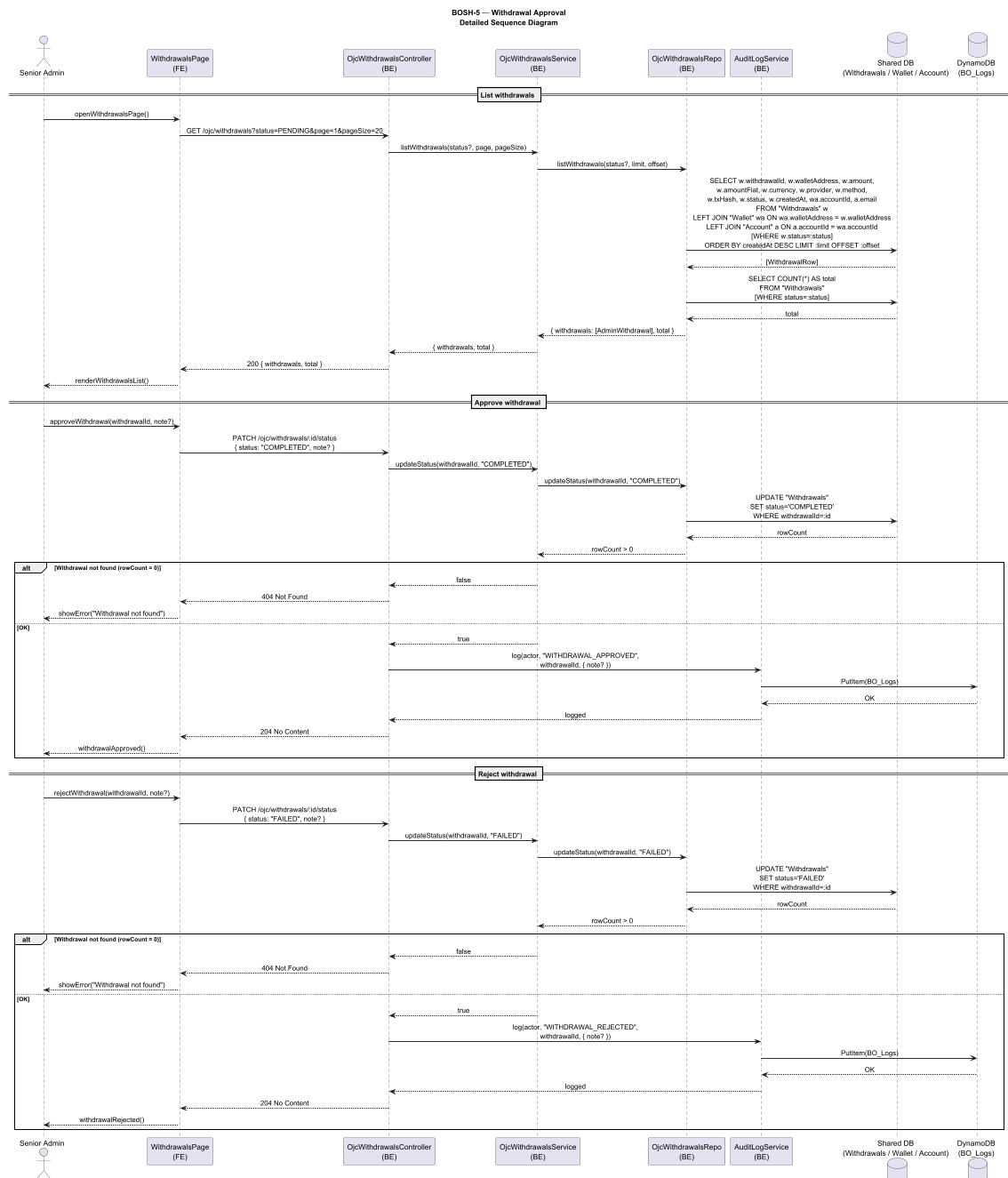


Figura A.4: SSD detalhado — Aprovação de levantamentos (BOSH-5)